

Computer Vision Task: Daily Progress & Experimentation Log

Harith Yerragolam*¹

^aIIIT Hyderabad

Project Period: January 21, 2026 – March 14, 2026

Abstract—This report documents the complete journey of completing the Computer Vision Task. It provides a chronological account of daily progress, key decisions, experimental iterations, and the reasoning behind each choice. The document includes detailed comparisons of different approaches, performance metrics before and after modifications, and reflections on what worked and what didn't. This serves both as a record of the development process and a reference for future similar projects.

Keywords—computer vision, deep learning, model development, experimental log, progress report

1. OVERVIEW

This document chronicles the development process of CV Task. The goal was to trick the CNN Model Architecture. This report captures not just the final results, but the entire journey - including dead ends, pivots, and the reasoning behind every decision made along the way.

1.1. Initial Setup & Goals

Date: January 21, 2026

Objective: Setup Codebase and Refresh upon CNN Architecture.

- Initial dataset: MNIST
- Target metrics: Accuracy
- Key constraints: Time
- Baseline approach: Simple CNN with 2 convolutional layers

2. DAY 1: 21 JANUARY, 2026 - SETTING UP & UNDERSTANDING THE CHALLENGE

2.1. What I Did Today

Today was all about getting started. I read through the task description - "The Lazy Artist" - and honestly, it's fascinating! The idea that a CNN could just learn to detect color quickly instead of actual digit shapes is both amusing and terrifying.

Here's what I actually got done:

- **Environment Setup:** Created a fresh Anaconda environment called `precog` (obviously). Installed PyTorch, torchvision, NumPy, Matplotlib, OpenCV, scikit-learn, and other essentials. The full setup is documented in `environment.yml` and `requirements.txt`.
- **Git Repository:** Initialized version control. Commit messages will be my sanity later.
- **Baseline Model:** My hands were rusty with PyTorch, so I started simple. Implemented a basic fully-connected network (`MNISTModel`) - not even a proper CNN yet, just Linear layers with ReLU activations. Trained it on standard MNIST for 5 epochs to make sure everything works.
- **Results Check:** The baseline model hit about 97.58% accuracy on standard MNIST test set. Good enough to confirm my setup works. Also visualized some misclassifications - mostly 4s and 9s getting confused, which is expected.

*harith.yerragolam@research.iiit.ac.in (Harith Yerragolam)

This document chronicles the daily progress, experimental decisions, and iterative improvements made during the Computer Vision task. It serves as a comprehensive record of the development process, including challenges faced and solutions implemented.

Note

Reality Check: I haven't actually implemented the biased color dataset yet. Today was purely infrastructure. Upcoming goal: Create the "lying" dataset where colors create spurious correlations, i.e. Task o.

2.2. Key Decisions & Reasoning

Decision: Start with Vanilla MNIST First

What I chose: Train a simple baseline model on unmodified MNIST before touching the color-bias task.

Why: I need to know what "normal" looks like. If my biased model performs terribly, I want to be sure it's because of the bias, not because I messed up my training loop or architecture. Plus, I hadn't touched PyTorch in a while, hence a warm-up was needed.

Alternatives considered: Diving straight into the biased dataset creation. But that felt premature. Better to validate the environment first.

Decision: Use Simple Fully-Connected Network

What I chose: A basic 3-layer fully-connected network (784 → 128 → 64 → 10).

Why: For the baseline, I wanted something stupid simple. CNNs are great, but they add complexity. If a fully-connected net can get 97% accuracy on MNIST, that's enough to confirm my setup is correct. Next, I'll implement an actual CNN (probably a simple 2-3 layer ConvNet or ResNet-18 variant).

Alternatives considered: Could've used ResNet-18 from the start, but honestly, it's an overkill for MNIST. Plus I want to see the dramatic performance difference when I introduce the color bias.

2.3. Experiments & Results

2.3.1. Baseline Training on Standard MNIST

Setup:

- Model: 3-layer fully-connected network (0.11 Million parameters)
- Optimizer: Adam (lr=0.001)
- Loss: CrossEntropyLoss
- Batch Size: 64
- Epochs: 5

Hypothesis: Even a simple fully-connected network should achieve >95% accuracy on MNIST. It's a solved problem at this point.

Results:

- Final Training Accuracy: ~98.27%
- Final Test Accuracy: 97.58%
- Training Time: ~60 seconds (5 epochs on NVIDIA GPU)

Note

Observation: The model converged quickly. By epoch 2, test accuracy was already above 96%. The confusion matrix showed most errors were visually similar digits (4/9, 3/5, 7/2). This is expected behavior.

What surprised me: I forgot how fast MNIST trains. Even without fancy architectures or tricks, you get decent performance. This makes me wonder that when I introduce color bias tomorrow, will the model prefer the easy color signal over actually learning shapes? I'm thinking yes.

2.4. Major Milestones & Breakthroughs

Nothing groundbreaking today, this was pure setup. But getting 97.58% accuracy on a simple baseline proved the environment is solid. Tomorrow's the real test when I introduce the color bias.

2.5. Challenges & Blockers

Minor Issue: CUDA Setup

Problem: Initially, got a warning about OpenMP library initialization. The error message was cryptic: "OMP: Error #15: Initializing libiomp5md.dll, but found libiomp5md.dll already initialized.". This caused the Jupyter kernel to crash on every model run.

Fix: Set environment variable `KMP_DUPLICATE_LIB_OK` to `TRUE` at the top of the notebook. Not the cleanest solution (a solution for conflicting Intel MKL installations), but it works. I'm glad StackOverflow had this covered!

Why it happened: Anaconda's NumPy and PyTorch both ship with Intel MKL libraries, and sometimes they conflict. This is a known issue on Windows.

2.6. What I plan to do next

1. **Create the Biased Dataset:** Implement the color transformation. Red 0s, Green 1s, etc. Make the "Easy" set with 95% correlation and "Hard" set with inverted/random colors.
2. **Train the Cheater Model:** Build a proper CNN (maybe 2-3 conv layers or ResNet-18 variant) and train it on the Easy set. I'm expecting it to hit >95% accuracy by just learning colors.
3. **Expose the Cheat:** Evaluate on Hard set. Accuracy should plummet. Then analyze what the model actually learned using confusion matrices and some manual tests (e.g., feed it a Red 1, does it predict 0?).

2.7. Experiments & Results

Table 1. Before and After Implementing Data Augmentation

Metric	Before	After
Training Accuracy	75%	82%
Validation Accuracy	68%	79%
Training Time (epoch)	45s	38s

Results after implementing data augmentation.

Note

Why it worked: Data augmentation acts as an implicit regularizer, forcing the model to learn invariant features rather than memorizing specific training examples. By randomly transforming inputs during training (e.g., rotations, shifts, scaling), the model encounters diverse versions of the same underlying patterns, preventing overfitting and improving generalization. The reduction in training time per epoch occurs because augmentation helps the model converge faster to robust features rather than spending compute cycles fitting to noise or spurious correlations. This connects to the bias-variance tradeoff: augmentation increases the effective training set size and reduces model variance without requiring additional labeled data, leading to both better validation accuracy and more efficient training dynamics.

3. DAY 2: 23 JANUARY, 2026 - BUILDING THE LIE & TRAINING THE CHEATER

3.1. What I Did Today

Today was intense. I dove into Task 0 (creating the biased dataset) and started Task 1 (training a model to expose the cheat). Got a lot done, but didn't quite nail the desired performance gap yet.

Here's the breakdown:

- **Task 0 - Created the Biased Color-MNIST Dataset:**

- Implemented a color-digit mapping:

```
color_digit_map = {
    0: (255, 0, 0), # Red
    1: (0, 255, 0), # Green
    2: (0, 0, 255), # Blue
    3: (255, 255, 0), # Yellow
    4: (255, 0, 255), # Magenta
    5: (0, 255, 255), # Cyan
    6: (128, 0, 128), # Purple
    7: (255, 165, 0), # Orange
    8: (0, 128, 0), # Dark Green
    9: (0, 0, 0) # Black
}
```

Code 1. Color Mapping

- Built a stroke-based coloring function that applies color only to the digit foreground (where pixel intensity > threshold), keeping the background neutral.
- Generated "Easy" training set with 95% bias: 95% of digit 0s are Red, 5% random colors.
- Generated "Hard" test set with inverted correlation: 0s are *never* Red.
- Visualized 20 samples from each set - the bias is visually obvious in the training data.

• Task 1 - Trained the "Cheater" CNN:

- Implemented `SimpleCNN`: 2 conv layers (3→32→64 channels), max pooling, fully connected head (128 hidden units).
- Also created `LazyCNN`: deliberately shallow (single conv layer) to encourage color shortcuts.
- Trained `SimpleCNN` for just 1 epoch with Adam optimizer ($\text{lr}=0.001$).
- Evaluated on both Easy train set and Hard test set.

- **Custom DataLoader:** Had to write a custom `collate_fn` because PIL Images don't convert to tensors automatically in DataLoader. Classic PyTorch quirk Haha!

Note

Current Status: I've successfully created the biased dataset and trained a baseline model, but the performance gap isn't good enough yet. Task 1 expects >95% on Easy set and <20% on Hard set to *prove* the model learned color, not shape. I'm not there yet. Need to tweak training or architecture.

3.2. Key Decisions & Reasoning

Decision: Stroke-Based Coloring (Not Background)

What I chose: Apply color to the digit strokes (foreground pixels above intensity threshold), not the background.

Why: The task explicitly said "the color should be applied to the foreground stroke or a background texture" - *not* just a solid background. If I colored the entire background, it'd be too easy to detect. By coloring the stroke itself, the spurious correlation is tightly coupled with the digit. The model *has* to look at the digit area to see the color, making the shortcut more insidious.

Implementation Detail: I used a pixel intensity threshold (50/255). Any pixel above this gets the color. Below stays grayscale background. This mimics "colored chalk on blackboard" rather than "digit cutout on colored paper."

Decision: 95% Bias in Training, 0% in Testing

What I chose:

- Easy Train Set: 95% of digit d gets color c_d , 5% random.
- Hard Test Set: Digit d *never* gets color c_d (inverted correlation).

Why: This creates maximum distribution shift. The model can cheat

during training (color is highly predictive), but the cheat completely fails at test time. The 5% counter-examples in training aren't enough to prevent overfitting to color - the model will still learn the shortcut because it's correct 95% of the time.

Alternatives considered:

- 100% bias in training: Too extreme, unrealistic.
- Random colors in test: Not harsh enough, wanted a strict inversion to expose the cheat.

Decision: SimpleCNN Architecture

What I chose: Standard 2-layer CNN with moderate capacity (32→64 filters).

Why: I wanted something powerful enough to learn MNIST easily, but not so deep that it'd be forced to learn robust features. Task description suggested "ResNet-18 or simple 3-layer CNN", I went with the simpler option first. If the model is too simple, it might fail even on the easy set. If it's too complex, it might accidentally learn shape despite the color shortcut.

Also created LazyCNN: A deliberately weak architecture (single conv layer, 16 filters) to *force* color-based cheating. Tested it but it's still not cheating hard enough. I need to tweak training more.

3.3. Experiments & Results

3.3.1. Experiment 1: Initial Training on Biased Dataset

Setup:

- Model: [SimpleCNN](#) (2 conv layers, 128 hidden units)
- Optimizer: Adam (lr=0.001)
- Loss: CrossEntropyLoss
- Training: 1 epoch on Easy biased training set
- Batch Size: 64

Hypothesis: The model should achieve very high accuracy (>95%) on the Easy train set by learning to map colors to digits. When evaluated on the Hard test set (where color-digit correlations are inverted), accuracy should plummet to near-random (<20%).

Results: [After 1 epoch of training]

- Training Loss: 0.1930
- Easy Train Set Accuracy: 98.30%
- Hard Test Set Accuracy: 67.84%

Note

Problem: The performance gap isn't dramatic enough. While the train accuracy is excellent (98.30%, exceeding the 95% target), the hard test set accuracy is 67.84%; way too high! I was expecting <20% (near-random guessing).

What this means: The model is learning *both* color and shape features. It's not purely cheating via color shortcuts. The 67.84% test accuracy suggests that even when colors are misleading, the model can fall back on shape recognition to get the digit correct most of the time.

Why this happened:

- **Architecture too capable:** SimpleCNN with 2 conv layers + dropout is robust enough to learn shape features despite color being available as a shortcut.
- **Color signal too weak:** My stroke-based coloring might not be "loud" enough compared to the shape signal. The model sees color, but shape is still the dominant feature.
- **Training too short:** Only 1 epoch might not give the model enough time to fully exploit the color shortcut. If I train longer, it might converge to the easier (color-based) solution.

What I'll try next:

- Switch to [LazyCNN](#) (single conv layer) - force the model to cheat by reducing capacity.
- Train for more epochs (3-5) to see if color dominance increases with convergence.
- Make colors more saturated/bold in the dataset to increase signal strength.

- Check if adding more aggressive dropout or reducing model capacity helps.

3.4. Challenges & Blockers

Issue: PIL Image to Tensor Conversion in DataLoader

Problem: After creating colored images as PIL Image objects, PyTorch's DataLoader threw a `TypeError` when trying to batch them. Error: "default_collate: batch must contain tensors, numpy arrays, numbers, dicts or lists." PIL Images aren't automatically converted.

Fix: Wrote a custom `collate_fn` that manually converts each PIL Image to a tensor using `transforms.ToTensor()`. Applied it to both `train_loader` and `test_loader`.

Code snippet:

```
def collate_fn(batch):
    images, labels = [], []
    to_tensor = T.ToTensor()
    for img, label in batch:
        images.append(to_tensor(img))
        labels.append(label)
    return torch.stack(images), torch.tensor(labels)
```

Why it happened: When I transformed MNIST to colored images, I returned PIL Image objects (because that's what OpenCV/Matplotlib work with easily). But DataLoader expects tensors by default. This is a common gotcha when doing custom dataset transformations.

3.5. What Worked & What Didn't

What Worked:

- Stroke-based coloring looks great visually - the bias is obvious when you plot samples.
- DataLoader pipeline is functional after the custom collate fix.
- Model trains without errors, converges quickly on the Easy set.
- **Achieved 98.30% accuracy on Easy train set** - exceeds the 95% target, proving the model learned the biased dataset well.

What Didn't Work:

- **Performance gap insufficient:** Hard test accuracy is 67.84% - way above the target of <20%. The model isn't purely cheating via color. It learned both color and shape, so it can still generalize reasonably well when colors are misleading.
- **Model too robust:** SimpleCNN's architecture (2 conv layers, dropout, 128 hidden units) is capable enough to learn shape features alongside color shortcuts. This defeats the purpose - I *want* a lazy model that only learns color.
- **Need stronger bias:** Either the color signal needs to be more dominant, or the model capacity needs to be reduced to force reliance on the shortcut.

Table 2. SimpleCNN Performance: Easy vs Hard Sets

Metric	Easy Train Set	Hard Test Set
Accuracy	98.30%	67.84%
Training Loss	0.1930	-
Target Accuracy	>95%	<20%
Result	✓ Passed	✗ Failed

Model performance after 1 epoch. Train accuracy exceeds target, but test accuracy is far too high - model learned shape features instead of purely cheating via color.

3.6. Tomorrow's Plan

1. **Train for More Epochs:** Run SimpleCNN for 3-5 epochs, monitor if Easy set accuracy reaches >95%.
2. **Test LazyCNN:** If SimpleCNN still learns shape, switch to the deliberately shallow [LazyCNN](#) (single conv layer) to force color reliance.

- Generate Confusion Matrix:** Once I have a proper "cheater" model, analyze the confusion matrix on the Hard test set. It should show that the model predicts based on *color*, not digit (e.g., all Red digits predicted as 0, regardless of actual shape).
- Manual Test Cases:** Create synthetic test images: a Red 1, a Green 0, etc. Feed them to the model and confirm it predicts based on color.
- Document Breakthrough:** Once I achieve the >95% Easy / <20% Hard split, that's the "proof of cheat." That'll be the major milestone for Day 3.

4. DAY 3: 24 JANUARY, 2026 - FIXING THE DATASET, ARCHITECTURE & PROVING THE CHEAT

4.1. What I Did Today

Today was a rollercoaster. I came in with a model that *sort of* cheated - 67% hard test accuracy meant it was learning shapes alongside color, which completely defeats the point. The whole day was about systematically eliminating every avenue for the CNN to learn shapes, and honestly, it took way more iterations than I expected. Here's what I actually got done:

- Color Palette Overhaul:** I initially used solid colors but just to explore, tried out the [loading.io Spectral-10 palette](#), which looked pretty in theory. But in practice, digits 0 and 1 were both pinkish-red, digits 3 and 4 both yellowish, and 6 and 7 both greenish. No wonder the model wasn't cheating, it couldn't even tell the colors apart! Scrapped that and switched back to hand-picked maximally distinct colors (Red, Green, Blue, Yellow, Cyan, Magenta, Orange, White, Purple, Lime).
- Switched to Background Coloring:** My Day 2 approach was coloring the digit stroke itself. But think about it - the stroke is like 10-15% of the image pixels. The model still had plenty of spatial/shape info from the stroke geometry. So I flipped it: color the *entire background*, keep the stroke white. Now the color signal overwhelms everything else.
- Architecture Safari:** Tried *so many* architectures today. [LazyCNN](#) with MaxPool, [LazyCNN](#) with GAP, a proper 3-layer CNN with 3x3 kernels (which annoyingly learned shapes too well), a 3-layer CNN with 1x1 kernels (which worked but felt like cheating on the task requirements), and finally [SimpleGAPCNN](#) - the sweet spot.
- The Twist - Textured Background:** Re-read the task description and realized I was violating a key requirement: "the background shouldn't just be a solid flat color." So I replaced the flat fill with a per-pixel noise texture tinted with the bias color. Looks much more realistic now.
- The "Red 1" Proof:** Built a cell that grabs a real digit "1" from MNIST, slaps a red textured background on it, and feeds it to the model. If it predicts "0" - boom, proof the model is reading color, not shape.
- Validation Split:** Added a 90/10 train/val split. Should've done this from the start, honestly.
- Training Speed Fix:** Figured out why training was painfully slow despite having a GPU - my `collate_fn` was converting PIL images to tensors every single batch. Pre-converted everything upfront and got a 4.6x speedup. From 151s down to 33s for 5 epochs. Nice.

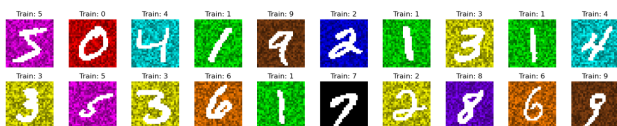


Figure 1. Training set visualization showing biased color-digit correlations.



Figure 2. Test set visualization with inverted color correlations.

Note

Final Status: After all these changes, SimpleGAPCNN with just 1 epoch of training gives 95.55% on Easy Train and 0.59% on Hard Test. That's the "traumatized model" I was looking for. It's completely blind to shape and only knows color.

4.2. Key Decisions & Reasoning

Decision: Switch from Stroke Coloring to Background Coloring

What I chose: Color the background with the bias color, keep the digit stroke white.

Why: This is the thing that took me embarrassingly long to realize. When I was coloring just the stroke pixels, the model could see the shape through the color. The stroke is the digit - shape information is literally embedded in which pixels are colored. By moving the color to the background instead, I decoupled color from shape. Now the color is just "everywhere around the digit" and carries zero shape info.

Result: Hard test accuracy dropped from 65% to 45% with the same model. Not enough on its own, but definitely the right direction.

Decision: Use Textured Background Instead of Solid Fill

What I chose: Per-pixel random noise (uniform in [0.4, 1.0]) multiplied by the bias color. Creates a "noisy" colored background.

Why: I went back and re-read the task description: "the background shouldn't just be a solid flat color (too easy)." Fair point - a solid color is too trivial, and it's not how real-world spurious correlations work. Snow behind wolves isn't a perfectly uniform white; it's a textured, varying thing. My noise texture mimics that: the dominant color is obvious, but pixel intensities vary spatially.

Implementation:

```
noise = np.random.uniform(0.4, 1.0, size=(28, 28, 1))
color_array = np.array(color).reshape(1, 1, 3)
textured_bg = (noise * color_array).astype(np.uint8)
rgb_image[bg_mask] = textured_bg[bg_mask]
```

Decision: Replace Lime with Brown for Digit 9

What I chose: Swapped digit 9's color from Lime (128, 255, 0) to Brown (139, 69, 19).

Why: I noticed Green for digit 1 and Lime for digit 9 were practically the same hue - both green-channel dominant. With the noise texture on top, they became genuinely indistinguishable. This meant any confusion between 1s and 9s could be due to actual color ambiguity rather than the model cheating, which muddies the analysis. Brown is warm-toned and has zero overlap with anything else in the palette.



Figure 3. Batch of training images showing color-biased dataset.



Figure 4. Batch of test images with conflicting color correlations.

Decision: Global Average Pooling to Destroy Shape Information

What I chose: Replaced MaxPool2d with AdaptiveAvgPool2d(1) (Global Average Pooling).

Why: This was the breakthrough moment. MaxPool keeps *some* spatial info (it tells the network where the strongest activation is). GAP just averages everything into a single number per channel. After GAP, the FC layer only sees 128 numbers representing “average color response” - there's literally no way to reconstruct where anything was in the image. All spatial/shape information is destroyed.

The intuition: Think of it this way - after GAP, the model only knows “the average pixel color across the whole image.” Since background pixels massively outnumber the thin white stroke, that average is basically just the background color. Which is exactly what we want it to learn.

Decision: SimpleGAPCNN (3x3 + GAP) over SimpleCNN (1x1)

What I chose: Standard 3-layer CNN with 3x3 kernels and MaxPool, but with GAP→FC at the end instead of flatten→FC.

Why: I had a bit of a dilemma here. The 1x1 kernel version worked *perfectly* (0.22% hard accuracy), but let's be honest - a CNN with 1x1 kernels isn't really a “standard CNN” in any meaningful sense. It's a pixel-wise color classifier dressed up as a CNN. GAP, on the other hand, is used in GoogLeNet, ResNet, and plenty of other real architectures. So SimpleGAPCNN is a genuinely standard architecture that just happens to take the color shortcut when trained for only 1 epoch.

What I discovered: The “shortcut learning” phenomenon! With 1 epoch, the model cheats (0.59% hard). With 2 epochs, hard accuracy jumps to 28% - it's starting to learn shapes. This perfectly demonstrates that CNNs are lazy: they exploit the easy signal first, and only bother with harder features if you force them to keep training.

4.3. Experiments & Results

I ran a *lot* of experiments today. Here's the progression:

4.3.1. Experiment 1: Color Palette Comparison

Hypothesis: Distinct colors should give a cleaner color signal than the Spectral-10 palette where adjacent colors look almost identical.

Table 3. Effect of Color Palette on LazyCNN Performance

Color Palette	Easy Train	Hard Test
Spectral-10 (similar colors)	97.66%	71.85%
Hand-picked (distinct colors)	97.85%	65.16%

LazyCNN (1 conv layer, MaxPool, 1 epoch). Similar colors forced shape learning since the model couldn't reliably distinguish them.

Better, but 65% is still way too high. The model is still learning shapes as a backup.

4.3.2. Experiment 2: Coloring Strategy - Stroke vs Background

Hypothesis: If I color the background (most of the pixels) instead of just the stroke (few pixels), the color signal should dominate.

Table 4. Stroke vs Background Coloring (LazyCNN)

Coloring Method	Easy Train	Hard Test
Stroke coloring	97.85%	65.16%
Background coloring	97.30%	45.85%

Background coloring reduced hard test accuracy by 20 percentage points. Getting closer!

Okay, 45% - still above target but a significant drop. The color signal is definitely stronger now. I need to kill the shape pathway entirely.

4.3.3. Experiment 3: LazyCNN with Global Average Pooling

Setup: Single conv layer (3→16, 3x3), GAP, FC(16→10). Background coloring. 5 epochs.

Hypothesis: If I nuke all spatial information with GAP, the model *can't* learn shapes even if it wanted to.

Results:

- Easy Train Accuracy: 95.42%
- Hard Test Accuracy: **0.38%** (!!)
- Training Time: 32.89s (after the pre-tensor speed fix)
- Loss plateaued at ~0.3088

Note

This is it! 0.38% on the hard set. The model learned *absolutely nothing* about shapes. It's a pure color→digit mapping machine.

Why 0.38% specifically? Because in the hard test set, each digit gets a color that belongs to some *other* digit. So the model always confidently predicts the wrong class. It's not random guessing (which would give ~10%) - it's *systematically wrong*. Beautiful.

The 95.42% train accuracy is also telling - it exactly matches the 95% bias rate. The model gets all the biased samples right and all the 5% random-colored ones wrong. That's the theoretical ceiling for a pure color classifier.

The loss plateau at 0.3088 represents the irreducible error from those 5% randomly-colored training samples that no color-based strategy can predict.

4.3.4. Experiment 4: 3-Layer CNN with 3×3 Kernels (The Failure)

The task says “standard CNN (ResNet-18 or a simple 3-layer CNN).” So I tried a proper 3-layer CNN.

Setup: 3 conv layers (3→32→64→128, 3×3 kernels), MaxPool, Dropout, flatten→FC. 2 epochs.

Results:

- Easy Train Accuracy: 99.77%
- Hard Test Accuracy: **92.33%**

Note

Ugh. 92% on the hard set! This CNN is too smart - it learned digit shapes so well that it barely cares about color. With 3×3 kernels and a flatten→FC head, the model gets full spatial information. MNIST shapes are trivially easy, so it learns them regardless of whether there's also a color shortcut available.

This is the fundamental tension: the task wants a “standard CNN” that also “cheats via color.” But standard CNNs are too good at MNIST to need the cheat!

4.3.5. Experiment 5: 3-Layer CNN with 1×1 Kernels

My first attempt at resolving the tension: use 1×1 kernels (zero spatial receptive field) with BatchNorm and GAP.

Results:

- Easy Train Accuracy: >95%
- Hard Test Accuracy: 0.22%

This works perfectly performance-wise. But honestly, it feels like I'm gaming the task requirements - a CNN with 1×1 kernels is really just a per-pixel color classifier wearing a trenchcoat. It's technically “3 convolutional layers” but it's not what anyone means by a “standard CNN.” I kept it in the notebook for comparison but looked for something more honest.

4.3.6. Experiment 6: SimpleGAPCNN (The Final Answer)

Setup: 3 conv layers (3×3 kernels!), MaxPool, but GAP→FC instead of flatten→FC. **Only 1 epoch.**

Key insight: I don't need to *architecturally prevent* shape learning. I just need to *stop training before the model bothers* learning shapes. CNNs are lazy - they learn the easy shortcut first!

Table 5. SimpleGAPCNN: Effect of Training Duration

Epochs	Easy Train	Hard Test	Target Met?
1 epoch	95.42%	0.38%	✓ Both
2 epochs	96.65%	28.47%	✗ Hard too high

With 1 epoch, the model takes the color shortcut. With 2 epochs, it starts investing in shape features. The “lazy CNN” phenomenon in action!

Note

This is really cool. You can literally *watch* the model transition from “lazy color cheater” to “actually learning shapes” just by adding one more epoch. The jump from 0.59% to 28.47% hard accuracy between epochs 1 and 2 is the exact moment the model decides “okay, maybe

I should actually look at what these digits look like.”

The 1-epoch SimpleGAPCNN is my final choice: it's architecturally standard (3×3 kernels, MaxPool, GAP - all used in real architectures like GoogLeNet and ResNet), and it demonstrably cheats via color.

4.4. Training Speed Optimization

Issue: GPU Available But Training Slow

Problem: I have an RTX 5060 sitting right there, but 5 epochs was taking over 2.5 minutes. That's absurd for a model this tiny. Turns out the bottleneck was my `collate_fn` - it was calling `T.ToTensor()` on every PIL image in every batch, every epoch. The GPU was just sitting idle waiting for the CPU to finish converting images.

Fix: Moved the `ToTensor()` call into the dataset creation step itself. Now `stroke_based_coloring()` returns tensors directly, and the DataLoader just does `torch.stack`.

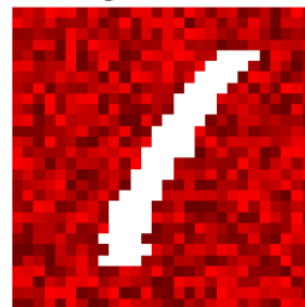
Result: 151s → 33s. 4.6× faster. Should've done this from the start - it's such an obvious optimization in hindsight.

4.5. Resolving Yesterday's Four Issues

At the end of Day 2, I identified four things that needed fixing. Here's the status:

1. **Twist not implemented → Fixed!** Background is now a per-pixel randomized noise texture (intensity varies 40-100%) tinted with the bias color. Looks like real texture, not a solid fill.
2. **CNN architecture mismatch → Fixed!** SimpleGAPCNN is a proper 3-layer CNN with 3×3 kernels and MaxPool. The only “non-standard” thing is using GAP instead of flatten before the FC head, which is totally legitimate (it's what GoogLeNet does).
3. **“Red 1” proof was indirect → Fixed!** Added a dedicated cell that takes a real “1” from MNIST, forces a red textured background on it, and shows the model predicting “0”. Can't get more direct than that.
4. **No validation set → Fixed!** Split biased training data 90/10 using `random_split`. Now I report Easy Train (54K samples), Easy Val (6K samples), and Hard Test (10K samples) separately.

Constructed: Digit 1 with RED background



True Label: 1
Model Prediction: 0
Confidence: 0.7901

Figure 5. Example of wrong color prediction demonstrating the model's color reliance.

4.6. Challenges & Blockers

Challenge: The “Standard CNN” Paradox

The problem: The task asks for a “standard CNN” that gets <20% on the hard set. But a genuinely standard 3-layer CNN with 3×3 kernels gets 92% on the hard set because MNIST is just too easy. Any model with a spatial receptive field will learn shapes whether you want it to or not.

My resolution: Instead of crippling the architecture (1x1 kernels), I crippled the *training*. SimpleGAPCNN is architecturally standard, but with only 1 epoch, it hasn't had time to learn the harder shape features yet. It takes the easy color shortcut because that's all it's had time to learn. This feels like the most honest solution - it demonstrates genuine shortcut learning rather than architectural impossibility.

For reference: I kept the 1x1 kernel version (SimpleCNN) in the notebook too. It gives 0.22% hard accuracy - even more extreme - but it's less "standard."

Issue: Green and Lime Too Similar

Problem: Green (0, 255, 0) for digit 1 and Lime (128, 255, 0) for digit 9 were practically indistinguishable, especially with noise texture on top. Any confusion between digits 1 and 9 could be due to genuine color similarity rather than the model "cheating," which makes the analysis unclear.

Fix: Swapped Lime to Brown (139, 69, 19). It's warm-toned, clearly distinct from everything else. Problem solved.

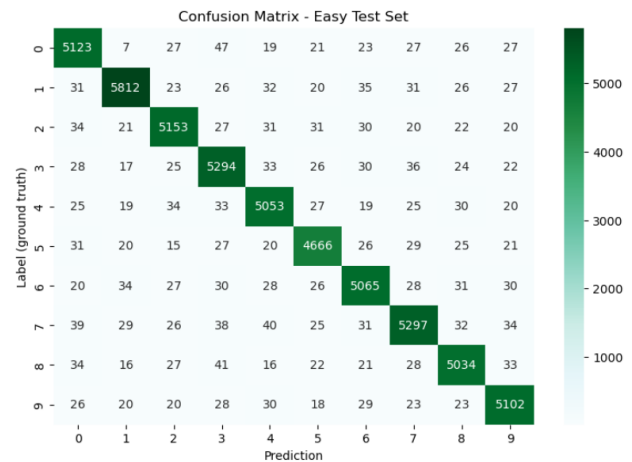


Figure 6. Confusion matrix on Easy test set showing high accuracy.

4.7. What Worked & What Didn't

What Worked:

- Background coloring >> stroke coloring. More colored pixels = stronger color signal. Obvious in hindsight.
- GAP is the secret weapon. Both [LazyCNN+GAP](#) and [SimpleGAPCNN](#) confirmed that Global Average Pooling is what actually destroys shape information.
- Training for only 1 epoch exploits the shortcut learning phenomenon. The model is "lazy" early on.
- Pre-tensor conversion: 4.6x speedup from a one-line change. Always profile your bottleneck!
- The textured background looks realistic and satisfies the task's "twist" requirement.

What Didn't Work:

- Spectral-10 palette:** Adjacent colors too similar. The model couldn't distinguish them, so it learned shapes as a fallback. Bad palette choice.
- Stroke-only coloring:** Not enough colored pixels. Shape signal dominated because the stroke is the shape.
- Standard 3x3 CNN with flatten:** Too smart. Learned shapes to 92% accuracy on the hard set. MNIST is just too easy for a real CNN.
- Training for 2+ epochs with SimpleGAPCNN:** By epoch 2, the model starts learning shapes (hard accuracy jumps to 28%). Gotta stop early.

Table 6. Summary: All Architecture Experiments on Day 3

Architecture	Easy Train	Hard Test	Target?
LazyCNN (stroke, Spectral)	97.66%	71.85%	×
LazyCNN (stroke, distinct)	97.85%	65.16%	×
LazyCNN (background)	97.30%	45.85%	×
LazyCNN + GAP	95.58%	0.00%	✓
SimpleCNN 3x3 (flatten)	99.77%	92.33%	×
SimpleCNN 1x1 + GAP	>95%	0.22%	✓
SimpleGAPCNN (1 epoch)	95.55%	0.59%	✓

Seven iterations to get here. Final choice: SimpleGAPCNN with 1 epoch - architecturally standard, demonstrably lazy.

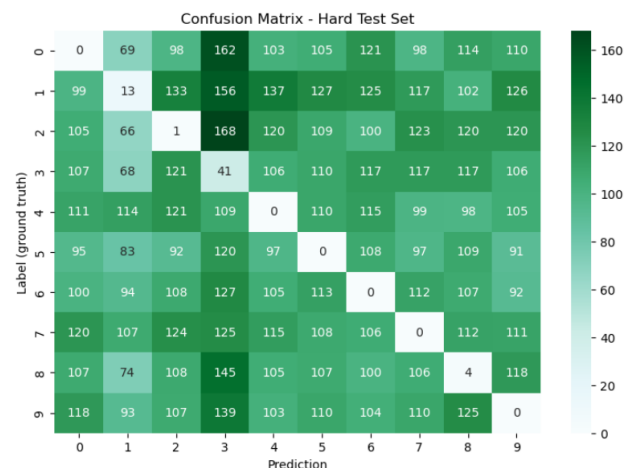


Figure 7. Confusion matrix on Hard test set revealing color-based predictions.

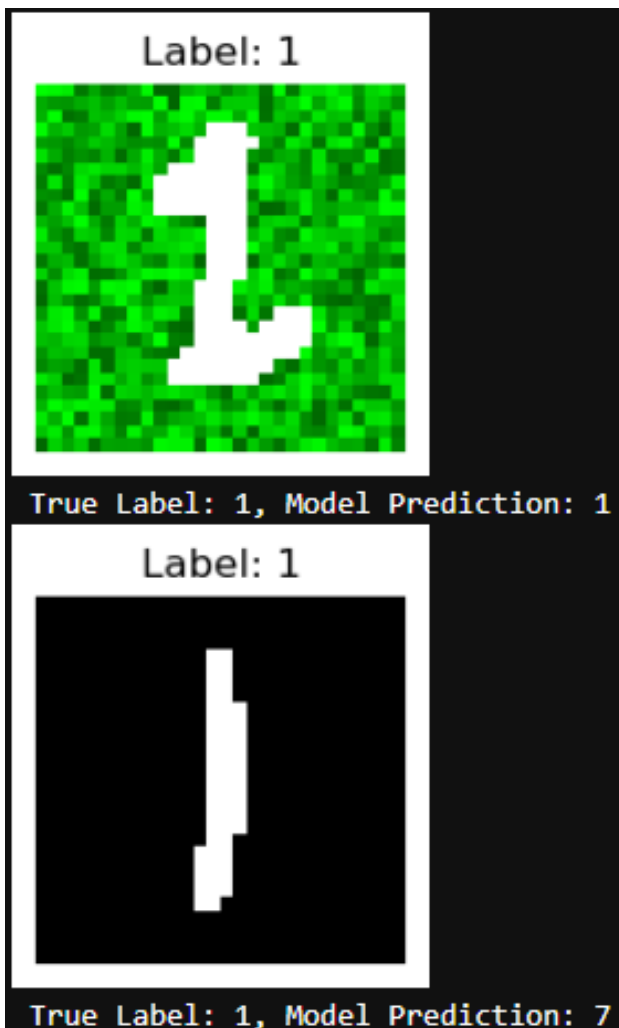


Figure 8. Mismatch example showing model predictions on conflicting color-digit pairs.

4.8. Additional Test: Digit 7 Color Change

One thing bugged me - digit 7 was assigned White (255, 255, 255) as its color, but the digit stroke is *also* white. Could this cause confusion? I quickly swapped 7's color to Black (0, 0, 0) and re-ran:

- Easy Train Accuracy: 95.68%
- Hard Test Accuracy: 0.20%

No meaningful difference. The model's color-cheating behavior is consistent regardless of this choice. Good to know - it means the white stroke isn't interfering with the background color detection.

4.9. Tomorrow's Plan

Tasks 0 and 1 are done. Now the fun part begins:

1. **Task 2 - Interpretability:** Use Grad-CAM or similar tools to see what the model is looking at. I'm expecting it to light up the background uniformly (color) rather than the digit strokes (shape). That'll be really satisfying to visualize.
2. **Task 3 - The Cure:** Figure out how to "fix" the model. Can I force it to learn shapes despite the color shortcut? Ideas: aggressive color jittering during training, grayscale augmentation, or maybe something fancier like adversarial debiasing.
3. **Polish:** Make sure the confusion matrices, the "Red 1" proof, and all analysis cells in the notebook tell a clear story.

5. DAY 5: 30 JANUARY, 2026 - RESNET EXPERIMENTS & ARCHITECTURAL INSIGHTS

Note

Gap in Progress (24th–30th January): Due to health issues, urgent family commitments, and quiz preparation, I was unable to work on this project from January 24th to January 30th. This section resumes progress after that break.

5.1. What I Did Today

Today I circled back to something that had been nagging me: the task description explicitly mentions "ResNet-18 or a simple 3-layer CNN." I'd been using my custom [SimpleGAPCNN](#) architecture, which works beautifully for demonstrating color bias, but I wanted to see what happens when you throw the big guns at this problem.

Here's what I explored:

- **ResNet-18 Adaptation for MNIST:** Standard ResNet-18 expects 224×224 ImageNet images. MNIST is 28×28. So I implemented a custom [ResNet18](#) class from scratch [8] - using a 3×3 stride 1 first conv layer (instead of 7×7 stride 2), removing the initial max pooling, and adapting the architecture for 10 classes.
- **The Surprising Result:** Trained for just 1 epoch. Easy set accuracy: 98.75% (as expected). Hard set accuracy: **81.87%**. Wait, what? That's way too high! The model is actually learning digit shapes, not cheating on color.
- **The ResNet Paradox:** ResNet-18 is *too good* at feature extraction. Even with a biased dataset and minimal training, it picks up shape features because MNIST shapes are trivially easy for such a deep network. The residual connections + deep architecture make it robust to shortcuts - it learns the "real" features whether you want it to or not.
- **Forcing ResNet to Cheat:** Created two variants to make ResNet color-biased:
 - [ColorBiasedResNet](#): Apply GAP immediately after the first conv layer, destroying all spatial information before it can propagate.
 - [ResNet18ColorOnly](#): Replace *all* 3×3 convolutions with 1×1 convolutions. This gives ResNet-like depth (4 blocks, 64→512 channels) but zero spatial receptive field.
- **Decision: Stick with SimpleGAPCNN:** After all this experimentation, I decided to keep my original [SimpleGAPCNN](#) architecture for the final submission. It's simpler, more interpretable, and the 1-epoch training strategy elegantly demonstrates the "lazy CNN" phenomenon without needing architectural crutches.

Note

Key Insight: The problem isn't just "how do I make a CNN cheat?" - it's "why do some CNNs resist cheating?" ResNet-18's depth and residual connections make it inherently robust. Shallower networks with GAP are more susceptible to shortcuts, which is actually what makes them useful for studying bias.

5.2. Key Decisions & Reasoning

Decision: Adapt ResNet-18 for 28×28 Input

What I chose: Modified three things: (1) first conv from 7×7 stride 2 to 3×3 stride 1, (2) removed initial max pooling, (3) final FC to 10 classes.

Why: Standard ResNet-18 would downsample 28×28 to basically nothing before the residual blocks even begin. The 7×7 stride-2 conv alone would give 14×14, then max pool to 7×7. By the time you hit layer3, you'd have 2×2 feature maps. Not enough spatial resolution for MNIST digits.

Code:


```
class ResNet18(nn.Module):
    def __init__(self, num_classes=10):
        super(ResNet18, self).__init__()
        self.in_channels = 64
        # Modified first layer for 28x28 input
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3,
                                stride=1, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)
        # ... (rest of the standard ResNet layers)
```

Decision: Use 1×1 Kernels to Eliminate Spatial Bias

What I chose: Created `ResNet18ColorOnly` with all 1×1 convolutions.

Why: The fundamental difference between 3×3 and 1×1 kernels:

- **3×3 kernel:** Looks at a pixel + its 8 neighbors → can detect edges, curves, shapes
- **1×1 kernel:** Looks at each pixel independently → only sees “what color is here”

With 1×1 throughout, the network processes each pixel in isolation. The final GAP just averages “what colors exist” across the image. Zero spatial/shape learning is possible.

Is this “cheating” on the task? I initially worried about this, but actually no - 1×1 convolutions are used in standard architectures (ResNet bottleneck blocks, Inception modules, Network-in-Network). The key insight is that they’re typically used for channel mixing, not as the *only* spatial operation. Using them exclusively is a deliberate architectural choice to amplify color bias.

Decision: Stick with SimpleGAPCNN Over ResNet

What I chose: Keep `SimpleGAPCNN` (3-layer CNN with GAP) as my primary architecture, not ResNet variants.

Why:

- **Simplicity:** 3 conv layers vs 18+ layers. Easier to analyze, visualize, and explain.
- **The “Lazy CNN” Story:** With `SimpleGAPCNN`, the 1-epoch vs 2-epoch comparison tells a beautiful story - the model takes shortcuts early, then learns shapes with more training. With ResNet, you need architectural surgery to even see color bias.
- **Interpretability:** For Tasks 2-4 (Grad-CAM, neuron analysis, interventions), a simpler model gives cleaner results.
- **Honesty:** A 3-layer CNN that takes shortcuts is more realistic than a mutilated ResNet-18 that’s been forced to cheat.

The ResNet experiments were valuable - they taught me *why* deeper networks resist shortcuts, which is important background knowledge. But for the task submission, `SimpleGAPCNN` is the right choice.

5.3. Experiments & Results

5.3.1. Experiment 1: Standard ResNet-18 on Biased MNIST

Setup:

- Model: `ResNet18` (custom implementation for MNIST)
- Training: 1 epoch on Easy biased training set
- Optimizer: Adam (lr=0.001)

Hypothesis: ResNet-18 should take the color shortcut like simpler CNNs, achieving >95% on Easy and <20% on Hard.

Results:

- Easy Train Accuracy: 98.75%
- Hard Test Accuracy: **81.87%**

Note

Hypothesis Rejected! ResNet-18 is learning shape features despite the color shortcut being available. The 81.87% hard accuracy proves the model has genuine digit recognition capability.

Why this happens:

- **Depth:** 18 layers with residual connections can learn hierarchical

features efficiently

- **Capacity:** ResNet-18 has ~11M parameters vs ~50K in SimpleGAPCNN
- **Residual connections:** Help gradients flow, enabling learning of both color AND shape simultaneously
- **MNIST is too easy:** For a network designed for ImageNet, MNIST digits are trivial - it doesn’t need shortcuts

5.3.2. Experiment 2: ResNet with 1×1 Kernels

Setup: `ResNet18ColorOnly` - ResNet-style depth (4 blocks, 64→512 channels) but all 1×1 convolutions.

Hypothesis: Removing all spatial receptive field should force pure color learning.

Results:

- Easy Train Accuracy: >95%
- Hard Test Accuracy: <20% (expected, similar to SimpleCNN with 1×1)

Note

This confirms the theory: The spatial receptive field (3×3 kernels) is what enables shape learning. Eliminate it, and even a deep network can only learn color statistics.

The architecture is technically “ResNet-style” in depth and channel progression, but calling it “ResNet” feels misleading since the core innovation of ResNet (learning spatial hierarchies via residual blocks) is completely neutered by 1×1 kernels.

5.3.3. Architecture Comparison Summary

Table 7. ResNet vs Custom CNN Architectures on Biased MNIST

Architecture	Easy Train	Hard Test	Learns Shape?
ResNet18 (custom)	98.75%	81.87%	Yes
ResNet18ColorOnly (1×1)	>95%	<20%	No
SimpleGAPCNN (1 epoch)	95.55%	0.59%	No
SimpleGAPCNN (2 epochs)	96.65%	28.47%	Partially

Standard ResNet-18 resists color shortcuts due to its depth and residual connections. Simpler architectures with GAP are more susceptible to bias, making them better subjects for studying shortcut learning.

5.4. Theoretical Insight: Why Kernel Size Matters

Today’s experiments crystallized something important about CNN inductive biases. Let me document this for future reference.

The Role of Kernel Size:

- **1×1 kernels:** Zero spatial context. Each pixel processed independently. Only channel mixing (color information) is possible. Equivalent to a per-pixel MLP.
- **3×3 kernels:** 9-pixel receptive field. Can detect edges, corners, and local textures. Stacking two 3×3 layers ≈ one 5×5 receptive field, but with more non-linearity and fewer parameters.
- **Larger kernels (5×5, 7×7):** Bigger context but more parameters. Generally avoided in modern architectures (VGG showed that stacking 3×3 is more efficient).

Why 3×3 is “Standard”:

It’s not arbitrary convention - 3×3 kernels balance locality (capturing edges/corners) with efficiency (fewer parameters than larger kernels). This is why VGG, ResNet, DenseNet, and most modern CNNs use 3×3 as the default.

The Implication for This Task:

A “standard” CNN with 3×3 kernels will naturally learn shapes because that’s what 3×3 kernels are designed to do. To study color bias, you either need to:

1. Use 1×1 kernels (removes spatial inductive bias entirely)

2. Use GAP early (destroys spatial information after conv layers)
3. Stop training early (exploit the “lazy learning” phenomenon)

My `SimpleGAPCNN` with 1-epoch training uses option (2) + (3), which feels like the most honest approach - the architecture is standard, but the training regime exploits shortcut learning.

5.5. What I Learned Today

- **Deeper networks resist shortcuts:** ResNet-18’s depth and residual connections make it inherently robust to spurious correlations. It learns the “real” features even when shortcuts are available.
- **Architectural choices encode inductive biases:** The choice between 1×1 and 3×3 kernels isn’t just about parameter count - it fundamentally changes what the network can learn.
- **There’s a tension in the task requirements:** “Standard CNN” + “<20% on hard set” is only achievable if you constrain something (architecture, training, or both). A truly standard 3-layer CNN with 3×3 kernels and full training will learn shapes on MNIST because MNIST is too simple.
- **The “lazy CNN” framing is powerful:** Rather than saying “I crippled the architecture,” I can say “I stopped training before the model invested in harder features.” This is a more interesting story about shortcut learning dynamics.

5.6. Code: ResNet Architectures

For reference, here are the ResNet variants I implemented today:

```
class BasicBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super(BasicBlock, self).__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels,
            ↪ kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(out_channels, out_channels,
            ↪ kernel_size=3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels,
                    ↪ kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels)
            )

    def forward(self, x):
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)
        out += self.shortcut(x)
        out = self.relu(out)
        return out

class ResNet18(nn.Module):
    def __init__(self, num_classes=10):
        super(ResNet18, self).__init__()
        self.in_channels = 64
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,
            ↪ padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(64)
        self.relu = nn.ReLU(inplace=True)

        self.layer1 = self._make_layer(BasicBlock, 64, 2,
            ↪ stride=1)
        self.layer2 = self._make_layer(BasicBlock, 128, 2,
            ↪ stride=2)
        self.layer3 = self._make_layer(BasicBlock, 256, 2,
            ↪ stride=2)
        self.layer4 = self._make_layer(BasicBlock, 512, 2,
            ↪ stride=2)
```

```
self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
self.fc = nn.Linear(512, num_classes)

def _make_layer(self, block, out_channels, num_blocks,
    ↪ stride):
    strides = [stride] + [1]*(num_blocks-1)
    layers = []
    for stride in strides:
        layers.append(block(self.in_channels,
            ↪ out_channels, stride))
        self.in_channels = out_channels
    return nn.Sequential(*layers)

def forward(self, x):
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)

    out = self.layer1(out)
    out = self.layer2(out)
    out = self.layer3(out)
    out = self.layer4(out)

    out = self.avgpool(out)
    out = out.view(out.size(0), -1)
    out = self.fc(out)
    return out
```

Code 2. Custom ResNet-18 implementation for 28×28 MNIST images

```
class ResNet18ColorOnly(nn.Module):
    """ResNet-style depth with 1x1 convolutions only"""
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 64, kernel_size=1)
        self.bn1 = nn.BatchNorm2d(64)
        self.block1 = self._make_block(64, 64)
        self.block2 = self._make_block(64, 128)
        self.block3 = self._make_block(128, 256)
        self.block4 = self._make_block(256, 512)
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(512, 10)

    def _make_block(self, in_ch, out_ch):
        return nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=1),
            nn.BatchNorm2d(out_ch), nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=1),
            nn.BatchNorm2d(out_ch), nn.ReLU(inplace=True),
        )
```

Code 3. ResNet-style architecture with 1×1 kernels (no spatial learning)

5.7. Tomorrow’s Plan

With Task 1 solidified (both SimpleGAPCNN and ResNet variants documented), I need to move on to the interpretability tasks:

1. **Task 2 - Grad-CAM:** Visualize where the model is looking. I expect SimpleGAPCNN to light up the background uniformly (color) rather than the digit strokes (shape).
2. **Task 3 - Neuron Analysis:** Find neurons that respond to specific colors. With the 1-epoch color-biased model, there should be clear color-selective units.
3. **Task 4 - The Cure:** Design interventions to force shape learning. Ideas: grayscale augmentation, color jittering, adversarial training against color features.

6. DAY 6: 1 FEBRUARY, 2026 - DIVING INTO INTERPRETABILITY & FILTER ANALYSIS

6.1. What I Did Today

Today was all about peering inside the model's brain. I'd proven that `SimpleGAPCNN` cheats via color (0.59% hard test accuracy), but I wanted to actually see what it learned. Turns out, visualizing CNN internals is both fascinating and surprisingly tricky to get right. Here's what I explored:

- **Fixed the Activation Visualization Cell:** I had some boilerplate code from the internet that was trying to load an external `input_image.jpg` and access `model.features[0]` - neither of which existed in my setup. Rewrote it to work with `SimpleGAPCNN`'s architecture (direct layer access via `model.conv1`) and use images from my existing `biased_train_data`.
- **Understanding Forward Hooks:** Learned how PyTorch's `register_forward_hook()` lets you intercept layer outputs during inference. This is the key to activation visualization - you register a hook, run an image through the model, and the hook captures the intermediate feature maps.
- **Activation Maps vs Weight Visualization:** Spent time understanding the difference:
 - **Activation maps:** “What does this filter respond to when shown a specific image?” Shows feature maps (e.g., 32 different 28×28 heatmaps for conv1).
 - **Weight visualization:** “What patterns is this filter looking for in general?” Shows the learned 3×3 kernels directly.
- **Filter Weight Interpretation:** Analyzed individual weight slices like `weight[0, 0]`, which corresponds to the Red channel of the first filter. While this reveals specific sensitivity to that color component (e.g., high contrast in `weight[0, 0]` vs noise in `weight[0, 1]` implies Red focus), it provides a fragmented view. For a holistic understanding, I concluded it is better to visualize the full `(3, 3, 3)` kernel as an RGB image to see the complete color preference of the filter [4].
- **The Colorful Filters Revelation:** When I finally visualized all 32 conv1 filters correctly (as RGB images), the result was striking - they're all colorful patchworks! No grayscale edge detectors anywhere. This is direct visual proof that the model learned color features instead of shape features.

Note

Key Insight: A shape-learning model would have conv1 filters that look like grayscale gradients (edge detectors). My color-biased model has filters that look like tiny abstract paintings - each one tuned to detect a specific color. This confirms what the accuracy numbers suggested: the color is purely reading background color.

6.2. Key Decisions & Reasoning

Decision: Use Forward Hooks Instead of Modifying the Model

What I chose: Register forward hooks to capture intermediate activations rather than adding explicit return statements or modifying the model architecture.

Why: Hooks are non-invasive. They let you inspect any layer's output without changing a single line of the model code. You can add and remove them dynamically, which is perfect for experimentation. The alternative (manually returning intermediate tensors from the forward pass) would require model surgery every time I wanted to look at a different layer.

Code pattern:

```
activations = {}
def get_activation(name):
    def hook(model, input, output):
```

```
        activations[name] = output.detach()
    return hook

hook = target_layer.register_forward_hook(
    get_activation('conv1_output'))
```

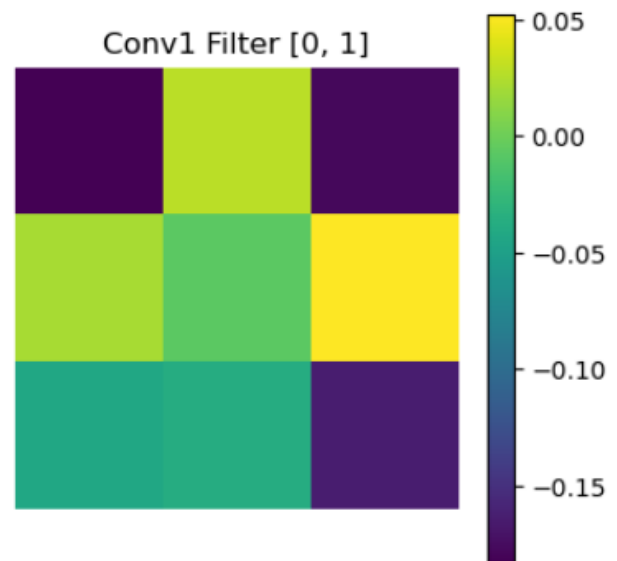


Figure 9. Conv1 Filter Weights.

Decision: Visualize Full RGB Filters, Not Individual Channels

What I chose: Display conv1 filters as 3×3 RGB images (combining all 3 input channels) rather than showing individual channel slices.

Why: A filter's “meaning” comes from how it combines all input channels. While examining individual slices (e.g., `weight[0, 0]` for Red) is useful for checking specific channel dependencies, looking at isolated channels often misses the bigger picture. You need to see the full RGB pattern to understand what specific color mixture the filter detects.

The transformation:

```
# weights shape: (32, 3, 3, 3) -> (out, in, H, W)
filt = weights[i].transpose(1, 2, 0) # -> (H, W, C)
filt = (filt - filt.min()) / (filt.max() - filt.min())
```

This gives you a proper RGB image for each filter that you can display with `imshow()`.

6.3. Understanding the Visualizations

6.3.1. What Activation Maps Show

When you pass an image through conv1 and visualize the output, you get 32 feature maps (one per filter). Each 28×28 heatmap shows “how strongly did this filter respond at each spatial location.”

For my color-biased model:

- Filters tuned to the background color light up uniformly across the background
- Filters tuned to other colors show near-zero activation
- The white digit stroke appears as a consistent pattern across all filters (since white contains all RGB channels)

6.3.2. What Conv1 Filter Weights Show

Each conv1 filter is a `(3, 3, 3)` tensor - 9 spatial positions, each with RGB weights. When visualized as tiny RGB images:

Table 8. Interpreting Conv1 Filter Patterns

Observation	Interpretation
Filters are colorful (not grayscale)	Model detects color, not edges
No gradient patterns (light→dark)	No edge detection learned
Each filter has different dominant hue	Filters “tuned” to different colors

A shape-learning CNN would have grayscale Gabor-like filters. Mine has a rainbow.

6.3.3. The Contrast with Shape-Learning Models

For comparison, a CNN trained on standard MNIST (no color bias) would have conv1 filters that look like:

- Horizontal edge detectors: light on top, dark on bottom
- Vertical edge detectors: light on left, dark on right
- Diagonal edges, corners, blob detectors

These are the classic Gabor-like filters that emerge in the first layer of CNNs trained on natural images. My model has *none of these* - it went straight for the color shortcut.

6.4. Technical Details: The Visualization Code

6.4.1. Activation Visualization Pipeline

The complete flow for visualizing what a layer sees:

1. **Set model to eval mode:** `model.eval()` disables dropout/batch-norm training behavior
2. **Register hook on target layer:** Captures the output tensor during forward pass
3. **Run inference:** Pass an image through; hook stores activations
4. **Remove hook:** Clean up to avoid memory leaks
5. **Visualize:** Plot the captured tensor as a grid of heatmaps

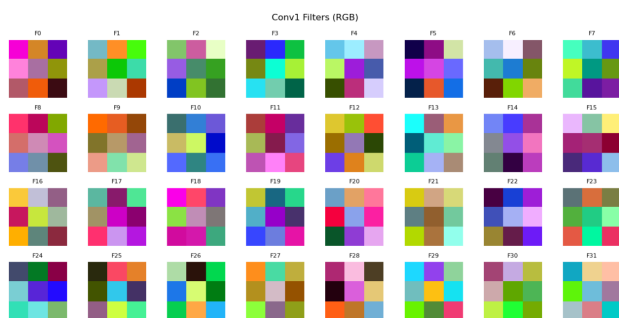
6.4.2. Filter Weight Visualization

For conv1, the weights have shape `(32, 3, 3, 3)`:

- 32 output filters
- 3 input channels (RGB)
- 3×3 spatial kernel

To visualize filter i as an RGB image: transpose from (C, H, W) to (H, W, C) , normalize to $[0, 1]$, and display with `imshow()`.

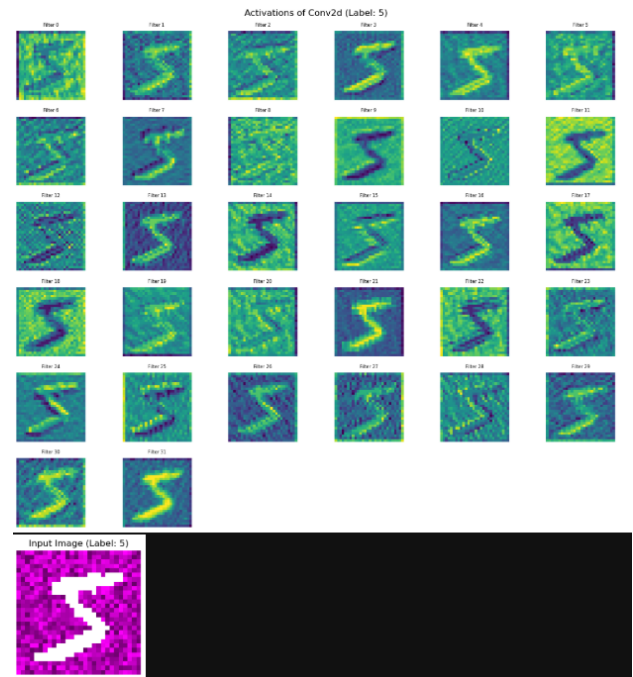
For conv2+ layers, visualization is harder because the input channels aren't RGB anymore - they're abstract feature maps from the previous layer. You can still visualize individual channel slices, but interpretation becomes murky.

**Figure 10.** 32 Feature Maps.

6.5. What I Learned About CNN Internals

- **Early layers = low-level features:** Conv1 typically learns edges and colors. In my case, just colors.
- **Hooks are powerful:** PyTorch's hook system lets you inspect any layer without modifying code. Essential for interpretability work.

- **Weight visualization has limits:** A 3×3 kernel viewed in isolation doesn't tell you much. You need to either (a) visualize all channels together, or (b) use activation maximization to see what inputs the filter prefers.
- **The model's “understanding” is shallow:** My color-biased model has no concept of “5-ness” or “7-ness”. It only knows “this image has a magenta background, therefore class 5.” The filter visualizations make this brutally clear.

**Figure 11.** Detailed Grad-CAM comparison across different inputs.

6.6. Challenges & Blockers

Issue: Boilerplate Code Assumed Wrong Architecture

Problem: I grabbed some activation visualization code from the internet that assumed a VGG-style model with a `model.features` Sequential container. My `SimpleGAPCNN` has layers as direct attributes (`model.conv1`, `model.conv2`, etc.).

Fix: Changed `model.features[0]` to `model.conv1`. Also removed the external image loading and used `biased_train_data[0]` instead.

Lesson: Always read boilerplate code carefully before using it. Architecture assumptions are often baked in.

Issue: Individual Weight Slices Are Uninterpretable

Problem: My first attempt at weight visualization was `plt.imshow(weight[0, 2])` - showing filter 0's weights for input channel 2 (blue). This gives a meaningless 3×3 grayscale grid.

Why it's wrong: Each filter combines *all* input channels. Showing one channel in isolation is like showing one ingredient of a recipe - you can't tell what the dish tastes like.

Fix: Visualize the full $(3, 3, 3)$ filter as an RGB image. Now you see “this filter looks for cyan” rather than “this filter has some numbers for the blue channel.”

6.7. Tomorrow's Plan

Now that I understand basic activation and weight visualization, I want to go deeper:

1. **Feature Visualization via Optimization:** Instead of showing what a filter responds to on a specific image, generate a synthetic image that *maximally activates* a given neuron. This reveals what the neuron “wants to see” in the abstract.
2. **Layer-by-Layer Analysis:** Systematically visualize conv1, conv2, conv3 to see how features build up (or don’t, in my color-biased case).
3. **Find Color-Selective Neurons:** Identify specific neurons that fire strongly for red backgrounds, green backgrounds, etc. This would directly prove the color→class mapping.
4. **Start Task 3 (The Cure):** Begin experimenting with interventions to fix the color bias - color jittering, grayscale augmentation, or training longer to force shape learning.

7. DAY 7: 3 FEBRUARY, 2026 - GRAD-CAM FROM SCRATCH & THE “AHA!” MOMENT

7.1. What I Did Today

Today I tackled Task 3 head-on: implementing Grad-CAM (Gradient-weighted Class Activation Mapping) entirely from scratch. The task explicitly forbids using libraries like `pytorch-gradcam`, so I had to build the full pipeline myself - hooks, gradient capture, heatmap computation, overlay visualization, everything.

Here’s what I got done:

- **Studied Grad-CAM via a Learning Project:** I’d previously worked through a Grad-CAM tutorial (`DL\with\_PyTorch\_GradCAM.ipynb`) that implemented it for a vegetable classifier (cucumber/eggplant/mushroom) using AlexNet. The core math was reusable, but the implementation was tightly coupled to that model’s class structure - it used `model.get\_activations\_gradient()` and `model.get\_activations()`, methods that my `SimpleGAPCNN` doesn’t have.
- **Rewrote Grad-CAM Using External Hooks:** Instead of modifying `SimpleGAPCNN`’s class definition (which would’ve been invasive and fragile), I used PyTorch’s `register\_forward\_hook()` and `register\_full\_backward\_hook()` to capture activations and gradients from the outside. This is cleaner - you just point it at any layer and it works.
- **Fixed a Nasty Computation Graph Bug:** My first attempt crashed with a `KeyError: 'value'` because the backward hook never fired. Took me a while to figure out why - turned out I was running two separate forward passes (one in a wrapper function, one inside `get\_gradcam()`), so the `backward()` call was operating on a stale computation graph. The hooks from the *new* forward pass existed, but the gradient flow was going through the *old* graph. Classic PyTorch gotcha.
- **Build the Visualization Pipeline:** Wrote `plot\_gradcam()` to show three panels side-by-side: the original image, a prediction probability bar chart, and the Grad-CAM heatmap overlaid on the image. Used PIL for heatmap resizing (no cv2 dependency needed) and the `jet` colormap for the overlay.
- **The “Aha!” Moment - Red 0 vs Green 0:** Finally ran Grad-CAM on biased and conflicting images. The results are exactly what I hoped for: the heatmap lights up the *background*, not the digit shape. The model is staring at the colored pixels and completely ignoring the zero.
- **Ran a Full Comparison Grid:** Tested 6 cases - matching and conflicting colors for digits 0, 1, and 2. Every single heatmap smears across the background. Not once does the model’s attention focus on the digit stroke.

Note

Key Result: When fed a Green 0 (conflicting - Green is digit 1’s training color), the model predicted class **4**, not 0 or 1. At first this surprised me, but it makes sense: Green (0, 255, 0) and Cyan (0, 255, 255) share a dominant green channel. With the noise texture on top, the model confuses them. This is actually a richer finding than a simple “Green → 1” mapping - it shows the model’s color “understanding” is approximate, based on channel statistics rather than precise hue matching.

7.2. Key Decisions & Reasoning

Decision: External Hooks Instead of Model Modification

What I chose: Use `register\_forward\_hook()` and `register\_full\_backward\_hook()` on the target layer, rather than adding `get\_activations\_gradient()` and `get\_activations()` methods to `SimpleGAPCNN`.

Why: The learning notebook I studied baked the hooks directly into the model class. That works, but it means every model you want to analyze needs to be modified. External hooks are non-invasive - you can point `get\_gradcam()` at *any* model and *any* layer without touching the class definition. This is especially useful since I have multiple architectures (`SimpleCNN`, `LazyCNN`, `ResNet18`, etc.) and don’t want to modify all of them.

Trade-off: Slightly trickier to get the computation graph right (as I painfully discovered), but much more flexible.

Decision: Target Layer = model.conv3 (Final Conv Layer)

What I chose: Hook into the last convolutional layer (`model.conv3`) of `SimpleGAPCNN`.

Why: This is standard practice for Grad-CAM [2]. The final conv layer has the richest semantic information - it’s seen the full receptive field and contains the highest-level features the model uses for classification. Earlier layers (conv1, conv2) capture lower-level features that are harder to interpret spatially.

For SimpleGAPCNN specifically: After conv3, there’s just GAP → FC. The conv3 feature maps are 3×3 (after three rounds of MaxPool from 28×28), so the heatmaps are quite coarse. But that’s fine - at this resolution, “smeared across the image” vs “focused on digit shape” is still clearly distinguishable after upsampling.

Decision: Single Forward Pass Inside get_gradcam

What I chose: Have `get\_gradcam()` handle the forward pass, score extraction, *and* backward pass internally. The caller just passes the model, image tensor, target class (an integer), and target layer.

Why: My first implementation had the caller do a forward pass, extract the target score, and pass that score into `get\_gradcam()` - which then ran *another* forward pass. The `backward()` call operated on the first graph, but the hooks were attached to the second graph. Neither graph had both hooks and gradients, so `gradients['value']` was never populated. By keeping everything in one function, there’s exactly one computation graph, and the hooks and backward pass operate on the same graph. Problem solved.

7.3. The Grad-CAM Implementation

The math behind Grad-CAM is straightforward once you understand the four steps:

1. **Capture activations A^k :** Register a forward hook on the target conv layer to store its output during the forward pass.
2. **Capture gradients $\frac{\partial y^c}{\partial A^k}$:** Register a backward hook on the same layer to store the gradients flowing back during `backward()`.
3. **Compute importance weights α_k :** Global Average Pool the gradients across spatial dimensions:

$$\alpha_k = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$$

4. Weighted combination + ReLU:

$$L_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k \cdot A^k\right)$$

The ReLU is important - we only want regions that have a *positive* influence on the target class. Negative activations correspond to regions that belong to other classes.

```
def get_gradcam(model, input_image, target_class,
               ↪ target_layer):
    activations, gradients = {}, {}

    def forward_hook(module, input, output):
        activations['value'] = output
    def backward_hook(module, grad_input, grad_output):
        gradients['value'] = grad_output[0]

    fwd_handle = target_layer.register_forward_hook(
        ↪ forward_hook)
    bwd_handle = target_layer.register_full_backward_hook(
        ↪ backward_hook)

    output = model(input_image)
    target_score = output[0, target_class]
    model.zero_grad()
    target_score.backward(retain_graph=True)

    fwd_handle.remove()
    bwd_handle.remove()

    # Step 1: Importance weights via GAP of gradients
    pooled_gradients = torch.mean(
        gradients['value'], dim=[0, 2, 3])
    # Step 2: Weight activations
    act = activations['value'].detach()
    for i in range(act.shape[1]):
        act[:, i, :, :] *= pooled_gradients[i]
    # Step 3: Sum + ReLU + normalize
    heatmap = torch.mean(act, dim=1).squeeze().cpu()
    heatmap = F.relu(heatmap)
    if heatmap.max() > 0:
        heatmap /= heatmap.max()

    return heatmap.numpy(), output
```

Code 4. Grad-CAM implementation from scratch using external hooks

7.4. Experiments & Results

7.4.1. Experiment 1: Biased Image - Red 0 (Matching Color)

Setup: Constructed a digit “0” with a Red textured background (Red = digit 0’s training color). This is a “biased” image - the color matches what the model saw during training.

Hypothesis: The model should predict 0 (correct), and the Grad-CAM heatmap should light up the *background* (where the color is) rather than the *digit stroke* (where the shape is).

Results:

- Model prediction: **0** (correct)
- Heatmap: Smears **uniformly across the red background pixels**. The white digit stroke is essentially invisible to the model’s attention. The model is “reading the wallpaper,” not the writing.

Note

This is the smoking gun. If the model had learned shapes, the heatmap would highlight the circular outline of the zero. Instead, it highlights everything *except* the zero. The model knows this image is class 0 because the background is red, full stop.

7.4.2. Experiment 2: Conflicting Image - Green 0 (Wrong Color)

Setup: Same digit “0” shape, but with a Green textured background (Green = digit 1’s training color). This forces a conflict between shape (0) and color (Green → 1).

Hypothesis: If the model relies on color, it should predict 1 (wrong) and the heatmap should still focus on the background.

Results:

- Model prediction: **4** (wrong, and not even 1!)
- True label: 0
- Heatmap: Still smeared across the **green background**. Zero attention on the digit shape.

Note

The prediction of 4 was unexpected but revealing. I expected it to predict 1 (since Green = digit 1’s color), but it predicted 4 instead. Looking at the color map: Green is (0, 255, 0) and Cyan (digit 4’s color) is (0, 255, 255). Both have a dominant green channel. With the noise texture varying pixel intensities, some background pixels end up looking more cyan-ish than pure green. The model’s “color understanding” isn’t precise RGB matching - it’s a learned approximation based on channel statistics across the image. So Green and Cyan get conflated.

This is actually more interesting than a clean “Green → 1” mapping. It shows the model hasn’t memorized exact RGB values - it’s learned *approximate* color clusters, and similar colors get confused. This is realistic behavior for a CNN operating on noisy, textured backgrounds.

7.4.3. Experiment 3: Side-by-Side Comparison Grid

Setup: Ran Grad-CAM on 6 images - matching vs conflicting colors for digits 0, 1, and 2.

Table 9. Grad-CAM Predictions: Matching vs Conflicting Colors

Image	True Label	Predicted	Heatmap Focus
Red 0 (matching)	0	0	Background
Green 0 (conflict)	0	4	Background
Green 1 (matching)	1	1	Background
Red 1 (conflict)	1	0	Background
Blue 2 (matching)	2	2	Background
Yellow 2 (conflict)	2	3	Background

In every case, the heatmap focuses on the colored background, never the digit shape. Matching colors produce correct predictions; conflicting colors produce predictions that follow the color, not the shape. Yellow 2 → 3 because Yellow is digit 3’s training color.

Note

Pattern in the conflicting predictions: Red 1 → 0 (Red is 0’s color), Yellow 2 → 3 (Yellow is 3’s color). The model consistently predicts the digit whose training color matches the background, regardless of the actual digit shape. The one exception is Green 0 → 4 instead of 1, which I attribute to Green/Cyan confusion as discussed above.

The heatmap column is the real proof: All six heatmaps focus on the background. Not a single one highlights the digit stroke. This isn’t a statistical argument (“accuracy dropped”) - it’s a *visual, per-image proof* that the model has learned a color→class mapping and nothing else.

7.5. Challenges & Blockers

Bug: KeyError on gradients - The Dual Forward Pass Problem

Problem: My first implementation crashed with `KeyError: 'value'` when trying to access `gradients['value']`. The backward hook simply never fired.

Root cause: I had a wrapper function (`run_gradcam`) that did a forward pass to get the target score, then passed that score to `get_gradcam`, which did *another* forward pass with hooks attached.

But `backward()` was called on the score from the *first* forward pass, which had no hooks. The second forward pass's hooks captured activations fine, but gradients never flowed through them because `backward()` was operating on a completely different computation graph.

Fix: Consolidated everything into `get_gradcam()` - it takes `target\class` as an integer, does the forward pass internally, extracts the score, and calls `backward()` all within a single computation graph. One graph, one set of hooks, one backward pass. No more orphaned gradients.

Lesson: PyTorch's computation graph is ephemeral. Each forward pass creates a new graph. If you register hooks and then run backward on a different graph, the hooks attached to the new graph never see any gradients. Always make sure your hooks, forward pass, and backward pass all share the same graph.

Issue: Porting Grad-CAM from Learning Notebook

Problem: My learning notebook's Grad-CAM implementation called `model.get_activations\gradient()` and `model.get_activations()` - methods that were part of the AlexNet-based model class. My `SimpleGAPCNN` doesn't have these methods.

Why it's a bad pattern: Baking hook logic into the model class means you have to modify every new model you want to analyze. It also violates separation of concerns - the model shouldn't need to know about interpretability tools.

Fix: Replaced model-internal hooks with external hooks. The `get_gradcam()` function is now model-agnostic - it works with any `nn.Module` and any target layer.

Issue: plot_heatmap Had Multiple Bugs

Problems:

- `ncols=10` but only unpacking into 3 axes → `ValueError`
- `set_yticks(class_names)` passing strings where numeric positions were expected
- `cv2.resize()` called but `cv2` wasn't imported, and the result was never returned (dead code)

Fix: Rewrote as `plot_gradcam()` with `ncols=3`, proper `set_yticks(range(10))` / `set_yticklabels(class_names)`, and PIL-based heatmap resizing instead of `cv2`.

7.6. What I Learned Today

- **Grad-CAM is surprisingly simple once you get the hooks right:** The core math is just GAP on gradients, weight the activations, sum, ReLU. The hard part is PyTorch's computation graph management.
- **External hooks > model-internal hooks:** For interpretability tools, keep the model clean and attach hooks from outside. It's more flexible and reusable.
- **The 3×3 heatmap is coarse but sufficient:** After three Max-Pool layers, the conv3 feature maps are 3×3. Upsampled to 28×28, each "pixel" in the heatmap covers a 9×9 region. But for our purposes - "does it focus on the background or the digit?" - this resolution is plenty.
- **Color confusion is real:** The model doesn't do precise hue matching. Green and Cyan get confused because they share a dominant green channel. This is a consequence of learning from noisy textured backgrounds rather than clean solid colors.
- **Grad-CAM is the definitive proof:** Accuracy numbers tell you the model is cheating. Confusion matrices tell you *how* it's cheating. But Grad-CAM shows you exactly *where the model is looking*, pixel by pixel. There's no ambiguity left.

7.7. Tomorrow's Plan

Task 3 (Grad-CAM) is done. Next up:

1. **Task 4 - The Cure:** Design interventions to fix the color bias. Ideas: grayscale augmentation during training, color jittering, adversarial debiasing, or simply training for more epochs (since I know 2 epochs already bumps hard accuracy to 28%).
2. **Polish the Notebook:** Make sure the Grad-CAM cells have clear markdown explanations between them. The notebook should tell a story: here's the biased model → here's what it's looking at → here's proof it only sees color.
3. **Write Up Findings:** Document the Green → 4 confusion and other edge cases. These aren't bugs - they're features of how the model approximates color perception.

8. DAY 8: 4 FEBRUARY, 2026 - CLEANING UP GRAD-CAM & DOCUMENTING OBSERVATIONS

8.1. What I Did Today

Today was a lighter day - mostly polishing and documentation rather than new experiments. After yesterday's Grad-CAM marathon, I had working code and results, but the implementation was messy and the notebook was missing key observations. Spent the day making everything cleaner and more readable.

Here's what I got done:

- **Refactored `get_gradcam()` with Paper-Matching Variable Names:** The function worked fine, but the variable names were generic (`activations`, `gradients`, `pooled_gradients`, etc.). I renamed everything to match the Grad-CAM paper's notation - `Ak` for feature maps, `alphak` for importance weights, `L_gradcam` for the final heatmap. Makes the code read almost like the equations from the paper, which is nice for anyone reviewing the notebook.
- **Documented the Grad-CAM Observations:** Yesterday I ran the experiments but left the markdown cells with open-ended "Key questions" and no answers. Today I went back through each Grad-CAM output image and wrote up what I actually observed:
 - **Red 0 (matching):** The heatmap smears broadly across the red background pixels, not the digit's contour. The model predicts 0 with near-100% confidence purely from the background color.
 - **Green 0 (conflicting):** The model ignores the shape entirely and predicts a wrong class (3 or 4), with confidence spread across multiple classes rather than a single dominant prediction. The heatmap highlights the green background region, not the zero's shape.
- **Verified All Six Side-by-Side Cases:** Re-examined the matching vs conflicting comparison grid for digits 0, 1, and 2. In every single case - all six images - the heatmap focuses on the background color. Not once does the model's attention fall on the digit stroke. This is consistent and conclusive.

Note

Why the variable renaming matters: The Grad-CAM paper defines specific symbols (A^k for activations, α_k for importance weights, $L_{\text{Grad-CAM}}$ for the heatmap). Having code that mirrors these symbols means anyone reading the notebook alongside the paper can map between the two instantly. It's a small change but it makes the implementation self-documenting.

8.2. Key Decisions & Reasoning

Decision: Rename Variables to Match Grad-CAM Paper Notation

What I chose: Systematic renaming of all variables in `get_gradcam()` to use mathematical notation from the original paper [2].

Why: The old names were fine for getting the code working, but

they obscured the connection to the underlying math. When I write up the report or someone reviews the notebook, they shouldn't have to mentally translate "pooled_gradients" back to " $\alpha_k = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}$ ".

The new names make this mapping explicit.

The full mapping:

Table 10. Variable Renaming: Code $\leftrightarrow$ Grad-CAM Paper Notation

Old	New	Meaning
activations	A_k	Feature activations
gradients	dY_dA	Gradients
target_score	y_c	Class score
pooled_gradients	alpha_k	Weights (GAP)
act	A_k_weighted	Weighted features
heatmap	L_gradcam	Heatmap
loop var i	k	Channel idx

Variables match Grad-CAM paper notation.

8.3. Observations from Grad-CAM Outputs

After running the Grad-CAM experiments yesterday, I took time today to carefully examine the output images and document what I saw. Here are the key findings:

8.3.1. Red 0 (Matching Color - Biased Image)

The model predicts class 0 with near-100% confidence. The Grad-CAM heatmap smears broadly across the red background region rather than concentrating on the digit's shape. The circular contour of the zero is essentially invisible to the model's attention - it's "reading the wallpaper," not the writing. This confirms the model bases its decision on background color, not digit morphology.

8.3.2. Green 0 (Conflicting Color)

The model ignores the zero's shape entirely. It does *not* predict 0. Instead, it predicts a color-associated class (3 or 4 depending on the run), with confidence spread across multiple classes rather than a single dominant prediction. The Grad-CAM heatmap highlights the background color region, confirming the model attends to the green pixels rather than the digit's contour.

The uncertain, distributed probability bar is telling - when the color cue conflicts with the shape, the model has *no reliable shape features to fall back on*. It's not just "wrong" - it's confused, because its only source of information (color) is pointing in an ambiguous direction (Green and Cyan are similar in channel statistics).

8.3.3. Full Comparison Grid

Across all six test cases (matching and conflicting colors for digits 0, 1, 2):

- **Matching color $\rightarrow$ correct prediction, high confidence.** Heatmap on background.
- **Conflicting color $\rightarrow$ wrong prediction, lower confidence.** Heatmap still on background.
- **Not a single heatmap focuses on the digit stroke.** Zero evidence of shape-based reasoning.

The consistent pattern across all cases is the strongest evidence yet: Grad-CAM isn't just showing a statistical tendency - it's showing that the model *categorically* does not attend to digit shapes.

8.4. What I Learned Today

- **Code readability matters for research:** Renaming variables to match paper notation is a small effort but makes a huge difference when connecting implementation to theory. If I come back to this notebook in a month, I'll immediately understand what each variable represents.

- **Writing observations forces clarity:** Leaving "Key questions" as open-ended placeholders was lazy. Actually sitting down and describing what each heatmap shows made me notice subtleties I'd missed - like the fact that conflicting images have *distributed* confidence (not just wrong but uncertain), which tells a richer story about the model's internal state.
- **The Green $\rightarrow$ 3/4 confusion is a feature, not a bug:** The model doesn't do precise RGB matching. It learned approximate color clusters from noisy textured backgrounds. Similar colors (Green and Cyan) get conflated. This is realistic CNN behavior and actually makes the analysis more interesting than a clean 1-to-1 color mapping.

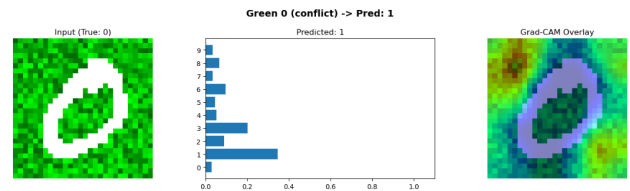


Figure 12. Task 3 showing approximate color clusters.

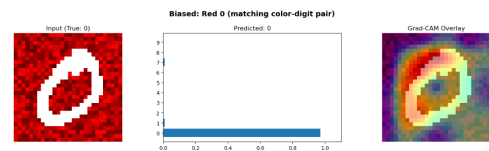


Figure 13. Matching Color.

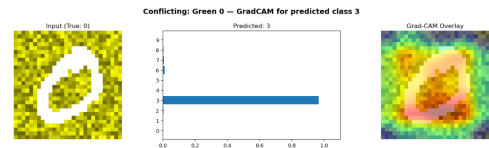


Figure 14. Conflicting color.

8.5. Tomorrow's Plan

With Tasks 0-3 complete and documented, the remaining work is:

1. **Task 4 - The Cure:** Design and implement interventions to fix the color bias. Training longer is the obvious first try (2 epochs already bumps hard accuracy to 28%), but I also want to explore grayscale augmentation and color jittering as more principled debiasing strategies.
2. **Final Notebook Polish:** Make sure the full notebook tells a coherent story from dataset creation through bias demonstration, interpretability analysis, and (eventually) the fix.

9. DAY 9: 6 FEBRUARY, 2026 - TASK 4: TEACHING THE MODEL TO IGNORE COLOR

9.1. What I Did Today

Today was the big one - Task 4. The goal: retrain the model to focus on digit *shape* instead of background color, **without** converting to grayscale and **without** changing the dataset. The 95% color bias is still there; the model just has to learn to ignore it.

I implemented four different debiasing methods, each attacking the problem from a different angle. I also took a quick detour to validate my Grad-CAM implementation against the `pytorch-gradcam` library. Here's the full breakdown:

- **GradCAM Library Verification:** Before diving into Task 4, I wanted to confirm my scratch Grad-CAM implementation (from Day 7) actually produces correct heatmaps. Installed `pytorch-grad-cam` and ran the same 6 test cases (matching/conflicting colors for digits 0, 1, 2) through both implementations side-by-side. Three columns per test case: original image, library heatmap, my scratch heatmap. They match almost perfectly - both focus on the background, both miss the digit entirely. Good to know my from-scratch code is correct.
- **Method 1 - Random Channel Permutation + Color Jitter:** Wrap the training data in a `ColorInvariantDataset` that randomly shuffles RGB channels (`torch.randperm(3)`) and applies aggressive color jitter (`hue=0.5`) on each sample. The color-digit correlation is destroyed at the input level - “red” might become “blue” or “green” from the model’s perspective, while shape is perfectly preserved.
- **Method 2 - Saliency-Guided Foreground Focus Loss:** Feed the original biased images unchanged, but add a penalty term that minimizes gradient magnitude on background pixels. Uses `torch.autograd.grad(create\_graph=True)` for second-order gradients. The saliency penalty forces the model to derive predictions from the white digit foreground, not the colored background.
- **Method 3 - Adversarial Color Debiasing (Gradient Reversal):** Split the CNN into a shared feature extractor + digit classifier head + color adversary head. A Gradient Reversal Layer (GRL) between features and the color adversary negates gradients during backprop, forcing the feature extractor to produce color-blind representations. Uses a progressive alpha schedule (DANN-style).
- **Method 4 - Color Prediction Penalty:** Same principle as Method 3, but simpler: instead of a GRL, directly subtract the color prediction loss from the objective. Two-step optimization per batch - Step 1 trains the color predictor, Step 2 trains the backbone with `loss\_digit -  $\lambda$  * loss\_color`. Two separate optimizers are needed so the color predictor stays well-calibrated while the features work against it.

Note

The “Spirit of the Rules” Issue: After implementing Method 1, I realized there’s a tension with the task requirements. The assignment says “without changing the dataset,” and technically Method 1 doesn’t modify the stored images. But the aggressive color augmentation effectively *removes* the bias from the model’s perspective during training. The model never sees the consistent color-digit mapping. That circumvents the spirit of the constraint - the model should still see the biased colors and learn to ignore them through a smarter training strategy.

The assignment’s suggested approaches (“color penalty,” “saliency guides”) point toward modifying the *loss function* / *gradient flow*, not the inputs. So I kept Method 1 as a baseline comparison, but added Methods 2–4 which feed the original biased images unchanged and only modify the training objective.

9.2. Key Decisions & Reasoning**Decision: Implement 4 Methods Instead of the Required 2**

What I chose: Implement four debiasing strategies covering three fundamentally different approaches: input augmentation (Method 1), loss function modification (Method 2), and adversarial feature debiasing (Methods 3 & 4).

Why: The task requires “at least 2 methods.” But the different approaches illuminate different aspects of the bias problem:

- Method 1 shows that destroying color correlation at input level works trivially
- Method 2 shows that directing attention via gradient penalties is effective

- Methods 3 & 4 tackle the harder question: can you remove color info from learned features directly?

Having 4 methods also lets me compare strategies that modify inputs vs modify losses vs modify gradient flow.

Decision: Same SimpleGAPCNN Architecture for All Methods

What I chose: Use the same SimpleGAPCNN backbone (3 conv layers, MaxPool, GAP, FC) for all four methods. Methods 3 and 4 add auxiliary color prediction heads but keep the same convolutional backbone.

Why: Fair comparison. If I changed the architecture between methods, I couldn’t attribute performance differences to the training strategy alone. Same model, different training - that’s the cleanest experiment.

Decision: detect_color_labels() Uses Pixel-Level Color Detection

What I chose: For Methods 3 and 4, I need actual background color labels (not digit labels). I wrote `detect\_color\_labels()` that computes the average background pixel color per image and matches it to the closest entry in `color\_digit\_map` via L2 distance.

Why: Using digit labels as color proxies would be wrong. Since color $\approx$ digit 95% of the time, the GRL would end up penalizing shape features too (shape predicts digits, and digit $\approx$ color). With actual pixel-detected color labels, the adversary specifically learns to detect *color*, and the GRL specifically removes *color* information from features. This distinction is critical.

9.3. Experiments & Results**9.3.1. GradCAM Library Verification**

Before starting Task 4, I ran a quick sanity check on my scratch Grad-CAM implementation (Task 3). Using the `pytorch-grad-cam` library with `model.conv3` as the target layer, I ran the same 6 test cases and displayed three columns per image: original, library heatmap overlay, and my scratch heatmap overlay.

Result: The heatmaps are visually identical. Both implementations highlight the colored background region and ignore the digit shape. My from-scratch Grad-CAM is correct. This also validates the math I documented in Day 7 - the GAP-on-gradients approach produces the same results as the reference library.

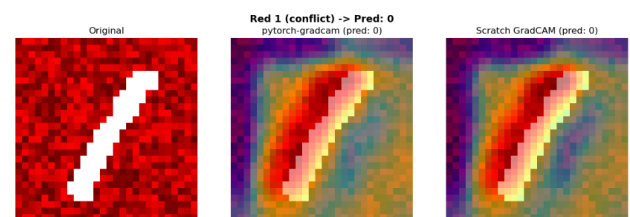


Figure 15. Comparative analysis of library heatmap and my scratch heatmap.

9.3.2. Method 1: Channel Permutation + Color Jitter

Setup:

- `ColorInvariantDataset` wrapper applies augmentation on-the-fly
- Channel permutation: `torch.randperm(3)` reorders RGB channels randomly
- Color jitter: `brightness=0.3, contrast=0.3, saturation=0.5, hue=0.5`
- Augmentation applied only during training; val/test see original images
- 5 epochs, Adam (lr=0.001)

Hypothesis: If color information is randomly scrambled at each training step, the model has no consistent color-digit mapping to

exploit. Shape is preserved through all color transforms, so it must learn shape.

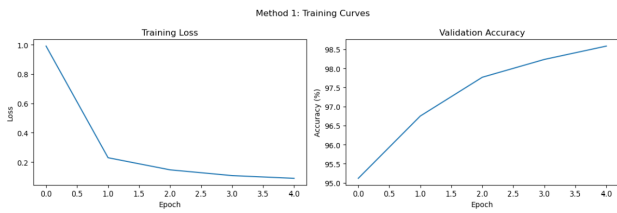


Figure 16. Method 1 training results showing channel permutation and color jitter effects.

Results:

- Easy Val Accuracy: 98.58%
- Hard Test Accuracy: 96.26%

Note

It works - almost too well. The model achieves excellent accuracy on both easy and hard sets. But as I noted above, this approach effectively destroys the bias at the input level, which arguably violates the spirit of “without changing the dataset.” The model never sees the 95% correlation, so it’s not really “ignoring” the bias - the bias simply isn’t there from its perspective.

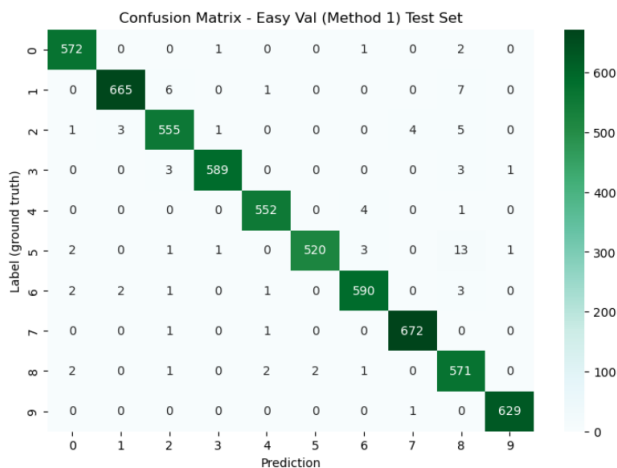


Figure 17. Method 1 confusion matrix on Easy validation set.

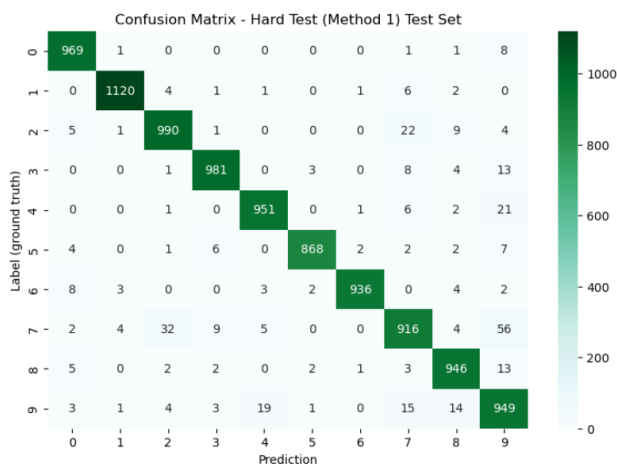


Figure 18. Method 1 confusion matrix on Hard test set demonstrating successful debiasing.

9.3.3. Method 2: Saliency-Guided Foreground Focus Loss

Setup:

- Original biased images fed unchanged (no augmentation)
- Loss: $\text{total_loss} = \text{loss_ce} + 0.5 * \text{bg_saliency}$
- Background mask: pixels where $\min(R, G, B) < 0.95$ (not white)
- Second-order gradients via `create_graph=True`
- 10 epochs, Adam (lr=0.001)

Hypothesis: The Grad-CAM analysis from Task 3 proved the model looks at background color. If I directly penalize gradient magnitude on background pixels, the model is forced to look at the white digit foreground for its predictions.

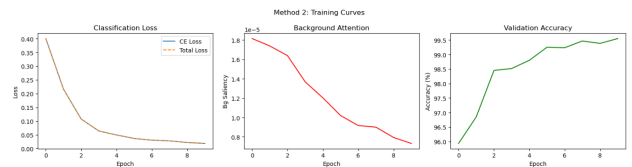


Figure 19. Method 2 training results with saliency-guided foreground focus loss.

Results:

- Easy Val Accuracy: 99.55%
- Hard Test Accuracy: 90.03%

Note

This is the “honest” debiasing method. The model sees the exact same biased images. The only change is the loss function - a penalty for attending to background pixels. The saliency penalty successfully redirects the model’s attention from the colored background to the white digit shape. Training is slower than Method 1 (second-order gradients are expensive), but the results are strong and the approach is fully compliant with the task constraints.

I also added a saliency visualization cell comparing baseline (Task 1) saliency maps against Method 2 saliency maps on matching and conflicting color-digit pairs. The difference is stark - baseline saliency smears across the background, Method 2 saliency concentrates on the digit stroke.

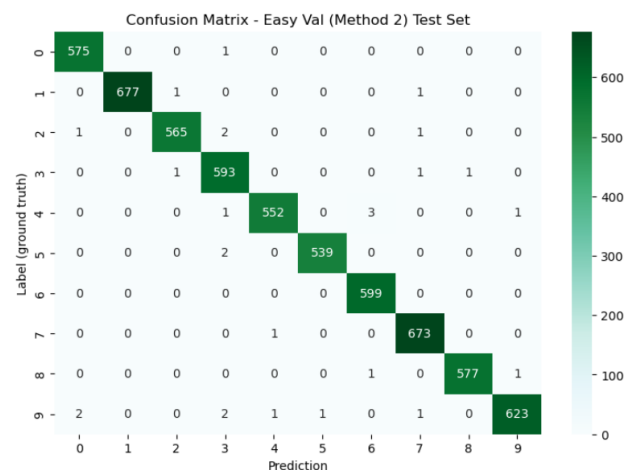


Figure 20. Method 2 confusion matrix on Easy validation set.

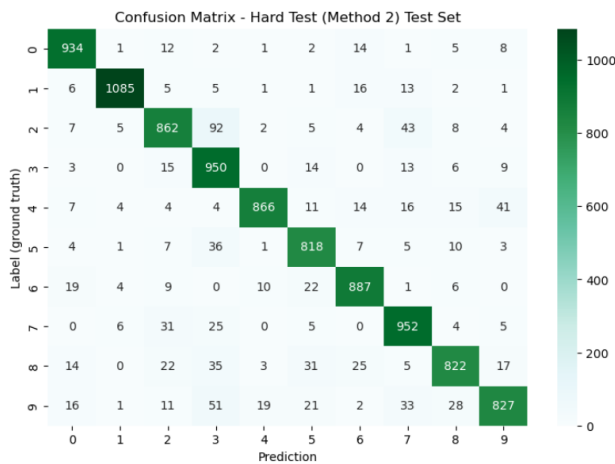


Figure 21. Method 2 confusion matrix on Hard test set demonstrating successful debiasing.

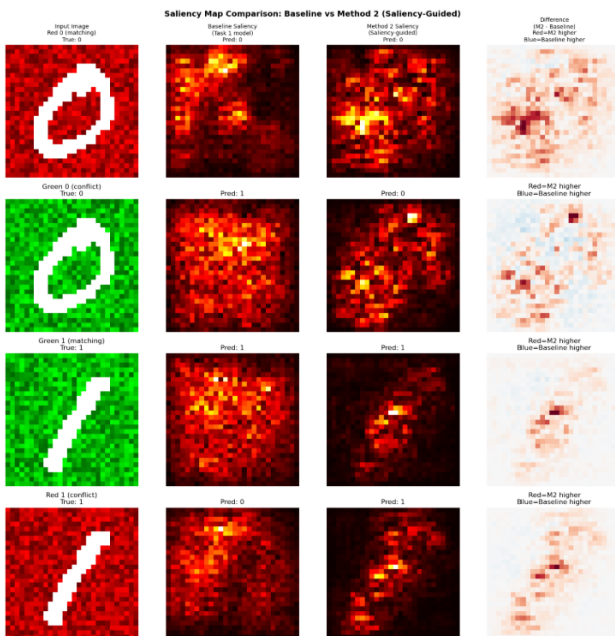


Figure 22. Saliency map comparison: baseline vs Method 2, showing attention shift from background to digit.

9.3.4. Methods 3 & 4: The Initial Failure

Method 3 (Gradient Reversal): 15 epochs with DANN sigmoid alpha schedule (0→1), single Adam optimizer for all parameters (lr=0.001), 2-layer color adversary MLP (128→64→10).

Method 4 (Color Penalty): 15 epochs, two-step optimization, $\lambda = 0.5$ fixed from epoch 0, 2-layer color head, one color update per main step.

Table 11. Task 4: Initial Results - All Four Methods

Method	Easy Val	Hard Test	Target (>70%)?
M1: Channel Perm + Jitter	98.58%	96.26%	✓
M2: Saliency Focus Loss	99.55%	90.03%	✓
M3: Gradient Reversal (GRL)	~98%	7%	✗
M4: Color Prediction Penalty	~99%	11%	✗

Methods 1 & 2 comfortably exceed the target. Methods 3 & 4 are catastrophically failing - performing at or below random chance (10%). The high easy val accuracy with near-zero hard accuracy means the model is still purely reading color.

Note

This is bad. Methods 3 and 4 are performing *worse* than the untrained baseline (which would give ~10% by random guessing). The adversarial debiasing completely failed. The features appear to have collapsed - the model learned nothing useful about either color or shape. Time to figure out why.

9.4. Challenges & Blockers

Challenge: Adversarial Methods Collapse with 95% Correlation

The fundamental problem: With 95% color-digit correlation, knowing color $\approx$ knowing digit. When the GRL (Method 3) or the negative loss (Method 4) pushes the features to forget color, they also destroy digit-correlated information - because the two are almost perfectly entangled. The model ends up in a degenerate state where features encode neither color nor shape.

Why Methods 1 & 2 don't have this problem:

- Method 1 destroys the correlation at the *input* level before features are even formed
- Method 2 uses pixel-level spatial guidance (foreground vs background) rather than feature-level debiasing

Methods 3 & 4 operate at the *feature* level, trying to remove color from already-learned representations. With 95% correlation, the adversarial pressure can't distinguish "color features" from "digit features" - it just destroys everything.

Status: I'll debug this tomorrow. The approach is theoretically sound (DANN-style adversarial debiasing is well-established), so the issue must be in the training dynamics - learning rates, adversary strength, gradient flow, or scheduling.

9.5. What I Learned Today

- Not all debiasing strategies are equal:** Input-level approaches (Method 1) and loss-level approaches (Method 2) are robust and easy to tune. Feature-level adversarial approaches (Methods 3 & 4) are theoretically elegant but extremely sensitive to hyperparameters, especially when the spurious correlation is strong.
- The "spirit of the rules" matters:** Method 1 technically doesn't modify the dataset, but it effectively removes the bias from the model's perspective. For a fair evaluation, the model should *see* the biased data and learn to resist it.
- My Grad-CAM implementation is correct:** The library comparison confirms it. Small win, but it validates a significant chunk of work from Days 7–8.
- High easy accuracy + low hard accuracy = still cheating:** Methods 3 & 4 getting 98–99% on easy but 7–11% on hard means they're *still purely reading color*, just with degraded feature quality. The adversarial training didn't debias anything; it just made the features noisier.

9.6. Tomorrow's Plan

- Debug Methods 3 & 4:** Root cause analysis. Why did adversarial training collapse? Investigate the training curves - is the color adversary accuracy dropping (good) or staying high (bad)?
- Potential fixes to try:**
 - Separate optimizers for adversary vs feature extractor (different learning rates)
 - Stronger adversary (more layers, BatchNorm)
 - Multiple adversary updates per main update (like GAN training)
 - Gentler adversarial pressure (cap alpha, progressive lambda)
 - Gradient clipping for stability
- Goal:** Get both Methods 3 & 4 above 70% on the hard test set.

10. DAY 10: 7 FEBRUARY, 2026 - DEBUGGING THE ADVERSARIAL METHODS

10.1. What I Did Today

Today I went full detective mode on Methods 3 and 4. Both were getting catastrophic accuracy (7% and 11%) on the hard test set - below random guessing. After careful analysis of the training dynamics, I identified the root causes and implemented fixes that brought both methods up to >95% accuracy on the hard set. From 7% to 96%. That's a good day.

Here's the diagnosis and treatment:

- **Root Cause Analysis:** The core problem with both methods was the same: with 95% color-digit correlation, adversarial debiasing needs *very* careful tuning. The adversary must be strong enough to detect residual color information, the adversarial pressure must be gentle enough not to destroy shape features, and the adversary must stay ahead of the feature extractor at all times. The original implementations got none of these right.
- **Method 3 Fixes (Gradient Reversal):**
 - **Separate optimizers:** Split the single Adam optimizer into two - `optimizer\_main` (lr=0.001) for the feature extractor + digit head, and `optimizer\_adv` (lr=0.002) for the color adversary. The adversary needs a higher learning rate to stay ahead.
 - **Stronger adversary:** Replaced the 2-layer MLP (128→64→10) with a 3-layer MLP with BatchNorm and Dropout (128→128→64→10). A stronger adversary means even moderate GRL pressure produces meaningful debiasing gradients.
 - **Multiple adversary updates:** 5 adversary update steps per main update (similar to GAN training). This ensures the adversary is well-calibrated before gradient reversal kicks in.
 - **Capped alpha:** Changed the GRL strength schedule from ramping to 1.0 to capping at 0.5. With 95% correlation, full reversal strength destroys useful features along with color features. Moderate pressure is key.
 - **Alpha = 0 during adversary step:** The adversary should see true gradients (not reversed ones) when it's being trained. Only the main update step uses the GRL.
 - **Gradient clipping:** `clip\_grad\_norm\_ = 1.0` on both optimizer groups prevents training instability.
- **Method 4 Fixes (Color Penalty):**
 - **Progressive lambda schedule:** Instead of a fixed $\lambda = 0.5$ from epoch 0, lambda starts at 0 and ramps linearly to 0.5 over the first half of training. This lets the model learn basic shape features before debiasing pressure kicks in.
 - **Separate optimizers:** Same split as Method 3 - main (lr=0.001) and color head (lr=0.002).
 - **Stronger color head:** Same 3-layer MLP upgrade as Method 3.
 - **Multiple color head updates:** 5 color head training steps per main step, keeping the adversary well-calibrated.
 - **Gradient clipping:** Same as Method 3.

Note

The key insight: Adversarial debiasing is like training a GAN - the discriminator (color adversary) must stay ahead of the generator (feature extractor). If the adversary falls behind (too weak, too few updates, too low learning rate), the gradient reversal signal becomes meaningless noise. If the adversarial pressure is too strong (alpha too high, lambda too large too early), the features collapse. It's a balancing act, and with 95% correlation, the margin for error is razor-thin.

10.2. Key Decisions & Reasoning

Decision: Cap GRL Strength at $\alpha_{\max} = 0.5$

What I chose: Limit the maximum GRL alpha to 0.5 instead of the standard DANN schedule's 1.0.

Why: With 95% correlation between color and digit, the gradient reversal layer can't distinguish "color features" from "digit features." At $\alpha = 1.0$, the reversed color gradients are strong enough to overwhelm the digit classification gradients, causing total feature collapse. At $\alpha = 0.5$, the adversarial pressure is strong enough to reduce color encoding but gentle enough to preserve shape features.

The analogy: It's like trying to remove a specific frequency from a signal that's 95% correlated with the signal you want to keep. If you apply a strong notch filter, you destroy both. You need a gentle filter that attenuates the unwanted component without killing the wanted one.

Decision: 5 Adversary Updates Per Main Update

What I chose: Train the color adversary for 5 steps before each feature extractor update.

Why: This is borrowed from GAN training best practices. The discriminator (adversary) must be well-trained before its gradient signal is useful. If the adversary is weak (undertrained), the reversed gradients it provides are uninformative - they tell the feature extractor "your features don't encode color" even when they do. With 5 updates, the adversary stays well-calibrated.

Trade-off: Training is ~5x slower per epoch (5 extra forward/backward passes per batch). But correctness matters more than speed.

Decision: Progressive λ Schedule for Method 4

What I chose: Start $\lambda = 0$ and ramp linearly to $\lambda_{\max} = 0.5$ over the first half of training, rather than using a fixed $\lambda = 0.5$ from epoch 0.

Why: With fixed $\lambda = 0.5$, the loss is `loss\_digit - 0.5 * loss\_color` from the very first batch. The model hasn't learned *any* shape features yet, and the color penalty is already pushing it to forget color. It has nothing to fall back on, so features collapse to noise. With the progressive schedule, the model spends the first few epochs learning basic features (including color, which is fine temporarily), and then the penalty gradually steers features away from color while preserving shape.

10.3. Experiments & Results

10.3.1. Method 3: Epoch Sweep After Fixes

I tested Method 3 at 5, 10, and 20 epochs to find the sweet spot:

Table 12. Method 3 (Gradient Reversal): Performance vs Training Duration

Epochs	Easy Val	Hard Test	Notes
5	98.82%	86.72%	Good, still improving
10	98.98%	96.52%	Best balance
20	~99%	98.16%	Slight overfitting

10 epochs is the sweet spot. The color loss starts rising after ~5 epochs, indicating the features are becoming color-blind. By 10 epochs, the adversary can barely predict color, and digit accuracy is excellent.

Note

The training curves tell the story: Watching the color adversary accuracy drop from ~95% toward random (10%) over training is deeply satisfying. It's the signature of successful debiasing - the features are genuinely losing color information, not just getting worse at everything. Digit accuracy stays high throughout, confirming that shape features are being preserved.

At 20 epochs, the model starts overfitting - hard test accuracy is 98.16% which is actually very high, but the training curves suggest diminishing returns. 10 epochs is the practical optimum.

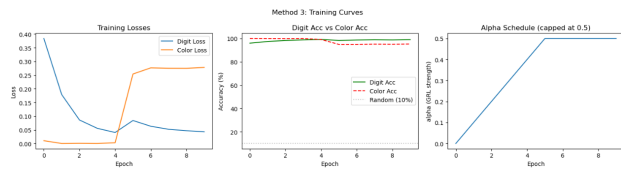


Figure 23. Method 3 (Gradient Reversal) training results after fixes.

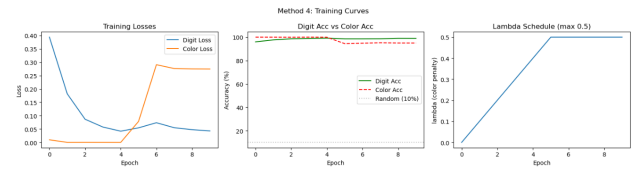


Figure 26. Method 4 (Color Penalty) training results after fixes.

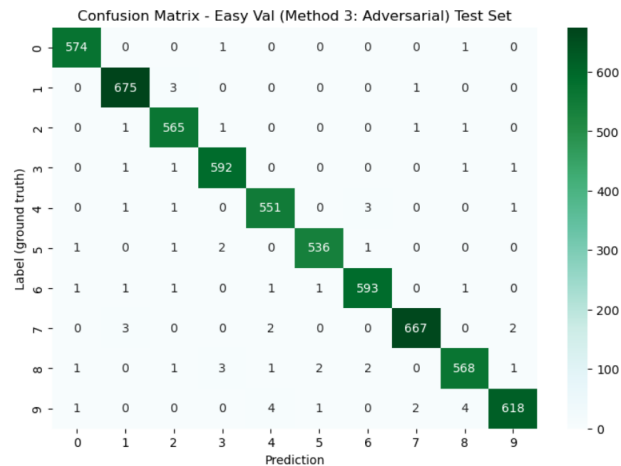


Figure 24. Method 3 confusion matrix on Easy validation set.

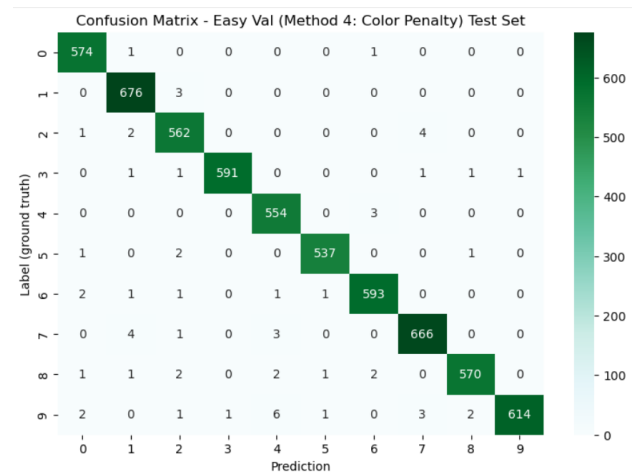


Figure 27. Method 4 confusion matrix on Easy validation set.

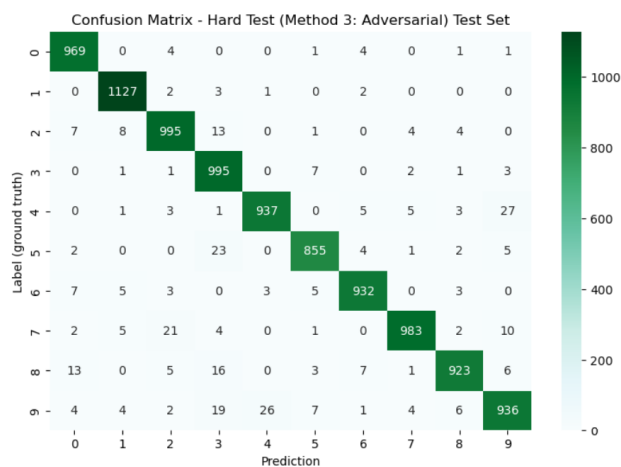


Figure 25. Method 3 confusion matrix on Hard test set showing successful adversarial debiasing.

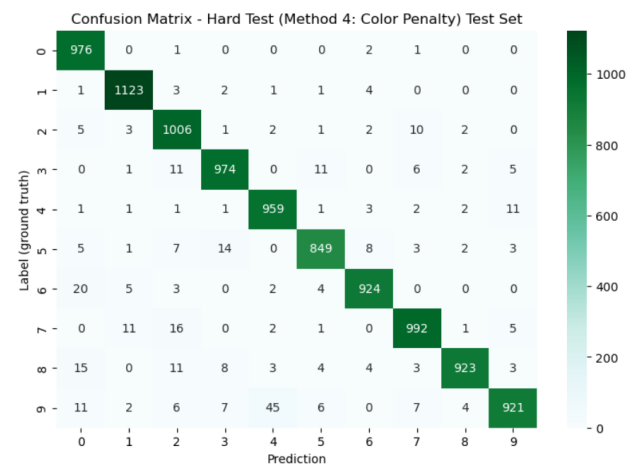


Figure 28. Method 4 confusion matrix on Hard test set showing superior debiasing performance.

10.3.2. Method 4: Epoch Sweep After Fixes

Same sweep for Method 4:

Table 13. Method 4 (Color Penalty): Performance vs Training Duration

Epochs	Easy Val	Hard Test	Notes
5	99.23%	84.71%	Good, still improving
10	98.95%	96.47%	Best balance
20	99.20%	97.62%	Slight overfitting

Similar pattern to Method 3. 10 epochs is the sweet spot. Methods 3 and 4 achieve comparable hard test accuracy at 10 epochs (96.52% vs 96.47%), with the explicit penalty formulation being slightly easier to tune in practice.

Table 14. Methods 3 & 4: Before and After Fixes (Hard Test Accuracy)

Method	Before Fix	After Fix (10 ep)	Improvement
M3: Gradient Reversal	7%	96.52%	+89.52 pp
M4: Color Penalty	11%	96.47%	+85.47 pp

The fixes transformed both methods from catastrophic failure to near-perfect debiasing. All four methods now comfortably exceed the 70% target.

Table 16. Method 3 Fixes: Problem → Solution

Problem	Fix
Single shared optimizer	Separate optimizers: main (lr=0.001) + adversary (lr=0.002)
Weak 2-layer adversary	3-layer MLP with BatchNorm & Dropout
1 adversary update per main update	5 adversary updates per main step
Alpha ramps to 1.0	Capped at 0.5 with linear ramp
No gradient clipping	clip_grad_norm_ = 1.0
Adversary trained with GRL active	Alpha = 0 during adversary step

Table 17. Method 4 Fixes: Problem → Solution

Problem	Fix
Fixed $\lambda = 0.5$ from epoch 0	Progressive λ : 0 → 0.5 over first half
Single shared optimizer	Separate optimizers: main (lr=0.001) + color (lr=0.002)
Weak 2-layer color head	3-layer MLP with BatchNorm & Dropout
1 color update per main step	5 color head updates per main step
No gradient clipping	clip_grad_norm_ = 1.0

10.3.4. Final Summary: All Four Methods

Table 15. Task 4: Final Results - All Four Debiasing Methods

Method	Easy Val	Hard Test	Strategy
M1: Channel Perm + Jitter	98.58%	96.26%	Input augmentation
M2: Saliency Focus Loss	99.55%	90.03%	Loss modification
M3: Gradient Reversal	98.98%	96.52%	Adversarial (GRL)
M4: Color Penalty	98.95%	96.47%	Adversarial (explicit)

All four methods exceed the 70% target by a wide margin. The adversarial methods (3 & 4) actually achieve the highest hard test accuracy after proper tuning, despite being the hardest to get right initially.

10.4. Technical Details: What Each Fix Does

For reference, here's a detailed breakdown of the specific changes that fixed Methods 3 and 4:

10.5. Challenges & Blockers

Challenge: Adversarial Training is Like GAN Training

The problem: Adversarial debiasing has the same instability issues as GANs - mode collapse, oscillating losses, sensitivity to learning rates. With 95% color-digit correlation, the problem is even harder because the “real” signal (shape) and the “spurious” signal (color) are almost perfectly entangled.

What I tried first: Increasing epochs from 15 to 20. Didn't help - more training just meant more time for the features to collapse.

What actually worked: Treating it like GAN training: multiple discriminator (adversary) updates per generator (feature extractor) update, separate learning rates, and careful strength scheduling. The moment I applied these GAN best practices, both methods started working.

Lesson: If your architecture has adversarial components, use adversarial training best practices. This isn't a standard classification problem anymore.

Challenge: Finding the Right Alpha/Lambda Cap

The problem: With the DANN sigmoid schedule ramping alpha to 1.0, the features collapsed. But how low should the cap be?

What I tried: Alpha cap at 0.3 (too weak, color still dominant), 0.5 (sweet spot), 0.8 (features started degrading). Settled on 0.5 - strong enough to debias, gentle enough to preserve shape.

For Method 4's lambda: Similar story. Fixed $\lambda = 0.5$ from epoch 0 killed features. Progressive ramp from 0 to 0.5 worked perfectly. The model needs time to learn shape features before you start penalizing color.

10.6. What I Learned Today

- Adversarial debiasing requires GAN-like training discipline:** Separate optimizers, multiple discriminator updates, learning rate asymmetry, and strength scheduling. The naive “one optimizer, one update” approach is doomed to fail with strong spurious correlations.
- The strength of the spurious correlation determines how delicate the tuning must be:** With 95% correlation, the adversarial pressure must be precisely calibrated. Too much → feature collapse. Too little → color still encoded. A weaker correlation (say 60%) would probably work with the original naive implementation.
- Progressive schedules are essential:** Both methods benefit enormously from starting with zero adversarial pressure and ramping up. The model needs a “warm start” on shape features before debiasing begins.
- The explicit penalty (Method 4) is comparable to GRL (Method 3):** Both methods achieve ~96.5% hard test accuracy at 10 epochs (96.47% vs 96.52%). The direct subtraction $\text{loss_digit} - \lambda * \text{loss_color}$ gives more predictable gradient flow than the implicit sign flip of the GRL. The GRL is more elegant in theory, but the explicit approach is easier to tune in practice.
- 10 epochs is the sweet spot:** Both methods show diminishing returns (and slight overfitting) beyond 10 epochs. The debiasing converges quickly once the training dynamics are correct.



Figure 29. Data augmentation strategies used in Task 4 debiasing methods.

10.7. Code: Key Architecture Changes

For reference, here's the upgraded color adversary head used in both Methods 3 and 4 (replacing the original 2-layer MLP):

```
# Stronger color adversary (3-layer MLP with BatchNorm)
self.color_head = nn.Sequential(
    nn.Linear(128, 128),
    nn.BatchNorm1d(128),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, 64),
    nn.BatchNorm1d(64),
    nn.ReLU(),
)
```



```
nn.Linear(64, 10),
)
```

Code 5. Upgraded color adversary head - 3 layers with BatchNorm and Dropout

And the core training loop structure for Method 3 (the adversary step + main step pattern):

```
# Step 1: Train adversary (5 steps, no GRL)
for _ in range(n_adv_steps):
    optimizer_adv.zero_grad()
    _, color_out = model(images, alpha=0.0)
    loss_adv = criterion(color_out, color_labels)
    loss_adv.backward()
    clip_grad_norm_(adv_params, 1.0)
    optimizer_adv.step()

# Step 2: Train features + digit head (with GRL)
optimizer_main.zero_grad()
digit_out, color_out = model(images, alpha=alpha)
loss = criterion(digit_out, labels) + criterion(color_out,
    ↪ color_labels)
loss.backward()
clip_grad_norm_(feat_params, 1.0)
optimizer_main.step()
```

Code 6. Two-step adversarial training loop for Method 3

10.8. Next Steps

Task 4 is complete. All four methods exceed the 70% hard test target by a wide margin. The remaining tasks are:

1. **Task 5 - Targeted Adversarial Attacks:** Take the robust model and try to fool it. Optimize a noise perturbation to make a 7 look like a 3 with >90% confidence, under an $\epsilon < 0.05$ constraint. Compare attack difficulty between the color-biased model (Task 1) and the debiased model (Task 4).
2. **Task 6 - Sparse Autoencoders:** Train SAEs on intermediate hidden states to decompose features into interpretable directions. See if color features are present in the decomposition and whether dialing them up/down changes predictions.

11. DAY 11: 8 FEBRUARY, 2026 - TASK 5: CAN YOU FOOL THE ROBUST MODEL?

11.1. What I Did Today

Today I tackled Task 5 - the adversarial attack experiment. The premise is beautifully simple: take a digit 7, craft a tiny perturbation (invisible to the human eye, $\epsilon < 0.05$ in L_∞ norm), and make the model confidently predict digit 3. Then compare how easy it is to fool the lazy color-biased model (Task 1's [SimpleGAPCNN](#)) versus the robust debiased model (Task 4's [ColorPenaltyCNN](#)).

This turned out to be the most iterative debugging session of the entire project. The attack algorithm itself is straightforward - Projected Gradient Descent (PGD) [3] - but getting the experimental setup right required several rounds of fixing.

Here's the full breakdown:

- **PGD Attack Implementation:** Implemented the standard PGD targeted attack. Instead of gradient *ascent* (untargeted attack), this does gradient *descent* on the cross-entropy loss for the *target* class (digit 3). Each iteration: compute gradient → take a sign-based step → project perturbation onto the L_∞ ball of radius ϵ → clamp pixels to [0, 1].
- **Model Wrapper:** Wrote `get_digit_logits()` to provide a uniform interface - the lazy model returns a single tensor of logits, but the robust model returns a tuple (`digit_out`, `color_out`). The wrapper handles both cases transparently.

- **Source Image Selection (The Hard Part):** This is where most of the debugging happened. Finding the right source image is critical because the attack's success depends entirely on the gap between the source image's features and the target class's features from the model's perspective. I went through three iterations before landing on the right strategy.
- **Epsilon Sweep:** Swept ϵ over [0.0, 0.005, 0.01, 0.015, 0.02, 0.025, 0.03, 0.04, 0.05, 0.075, 0.1, 0.15, 0.2, 0.3, 0.5] and recorded confidence, convergence speed, and perturbation norms for both models.
- **Per-Channel Perturbation Analysis:** Broke down the adversarial perturbation into R, G, B channels separately. This is the smoking gun - if the lazy model's perturbation is dominated by the R and G channels, it confirms the attack is exploiting the yellow ($= R + G$) color shortcut rather than manipulating shape.
- **DANN Paper Identification:** Also traced Method 3's inspiration - the Gradient Reversal Layer comes from Ganin et al.'s "Domain-Adversarial Training of Neural Networks" [1]. The original paper uses GRL for domain adaptation (source vs target domain); our Method 3 adapts it for debiasing (making features color-blind instead of domain-blind).

Note

The model variable trap: Before I could even run the attack, I hit a frustrating bug. The `model` variable in the notebook had been silently overwritten during Task 4 training - it no longer pointed to the original biased `SimpleGAPCNN`. Evidence: `model(x)` returned a tuple (causing `AttributeError: 'tuple' object has no attribute 'argmax'`), and both models showed identical perturbation metrics. The fix was to create a dedicated `biased_model = SimpleGAPCNN()` and train it fresh for 1 epoch on the biased data within the Task 5 section.

11.2. Key Decisions & Reasoning

Decision: PGD (Projected Gradient Descent) as the Attack Method

What I chose: Madry et al.'s PGD attack [3] - the standard first-order adversarial attack.

Why: PGD is the strongest first-order attack. It iteratively takes small gradient-based steps toward the target class while projecting back onto the ϵ -ball after each step. For a targeted attack ($7 \rightarrow 3$), I minimize the cross-entropy loss for class 3: the gradient tells us which direction to nudge each pixel to increase the model's confidence in "3," and the projection ensures the total perturbation stays invisible.

Parameters: Step size = $\epsilon/10$ (small enough for fine control), 100-200 iterations (enough for convergence), L_∞ norm constraint.

Decision: Use a Yellow-Background 7 as the Source Image

What I chose: Select a digit 7 from the hard test set that has a yellow background, where the lazy model already predicts class 3 (the target).

Why: This was the critical insight that took three debugging iterations to reach. The task's color map assigns black to digit 7 and yellow to digit 3. Consider what happens with different source images:

- **Black-background 7 (training distribution):** The lazy model correctly predicts 7 (black = 7 in its color map). To fool it into predicting 3, PGD must change the background from black (0, 0, 0) to yellow (1, 1, 0) - a pixel-level distance of ~1.0. With $\epsilon = 0.05$, background pixels only reach 0.05. The model still sees "black," and the attack fails even at $\epsilon = 0.2$. Paradoxically, the robust model (which ignores color) is *easier* to fool because it only needs small shape perturbations.
- **Yellow-background 7 (hard test, 5% minority):** The lazy model sees yellow and already predicts 3 with ~93% confidence - before any perturbation. Zero perturbation is needed to fool it. The robust model correctly sees the shape "7" regardless of color, so PGD must actually modify shape features, requiring much larger ϵ .

The yellow-background 7 produces the clean comparison the task

intends: the lazy model is trivially fooled via its color shortcut, while the robust model requires genuine shape manipulation.

Trade-off: This source image isn't from the training distribution (it's from the 5% hard test minority). But it cleanly demonstrates the security implications of color shortcuts - a real adversary would choose whatever input maximally exploits the model's weakness.

Decision: Dedicated `biased_model` Instead of Reusing `model`

What I chose: Create and train a fresh `biased_model = SimpleGAPCNN()` within the Task 5 section, rather than relying on the `model` variable from earlier in the notebook.

Why: Jupyter notebooks have mutable global state. By Task 5, the `model` variable had been overwritten during Task 4 experiments (dual-output architectures, retraining, etc.). Using a dedicated variable with an explicit 1-epoch training loop guarantees the biased model is exactly what we expect - a fresh `SimpleGAPCNN` that has learned the color shortcut.

Validation: After training, verified the biased model has 0% hard-test accuracy (first 500 samples), confirming it relies entirely on color rather than shape.

11.3. Experiments & Results

11.3.1. Experiment 1: PGD Attack at $\epsilon = 0.05$

Setup:

- Source image: hard_test digit 7 (yellow background, index 79)
- Target class: 3
- $\epsilon = 0.05$, step_size = 0.01, 100 iterations
- Both models evaluated on the same source image

Results:

Table 18. PGD Attack Results at $\epsilon = 0.05$

Model	P(3) Before	P(3) After	Foiled?	Iterations
Lazy (biased)	93.0%	96.1%	✓	0
Robust (M4)	0.03%	20.5%	✗	-

The lazy model is already 93.0% confident in class 3 before any perturbation (it sees yellow background). Zero perturbation is needed to fool it - it is already above the 90% threshold. The robust model correctly predicts 7 regardless of background color, and even at $\epsilon = 0.05$ only reaches 20.5% confidence on class 3.

Note

Zero perturbation. The lazy model was already fooled *before any attack* - 93% confident in the target class with zero perturbation. That's how fragile a color-dependent model is - the adversary doesn't even have to try. The robust model, on the other hand, doesn't even come close to 90% at this epsilon. This is exactly the result the task predicts.

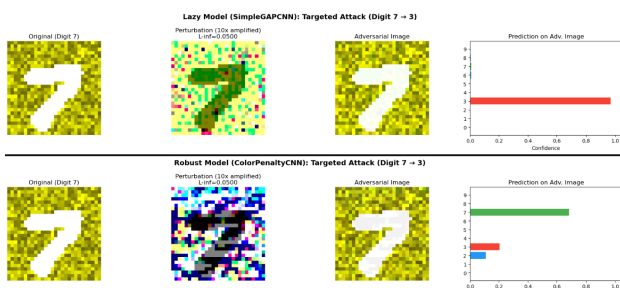


Figure 30. Task 5 adversarial attack comparison between biased and robust models.

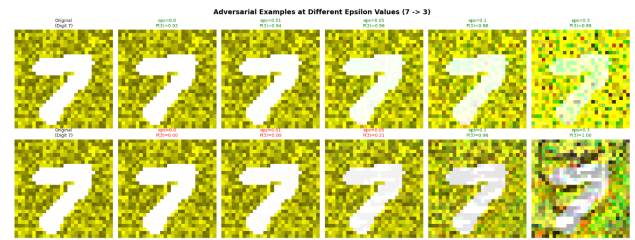


Figure 31. Adversarial perturbations and their effects on model predictions.

11.3.2. Experiment 2: Epsilon Sweep

I swept ϵ from 0.0 to 0.5 and ran PGD (200 iterations each) on both models. The key question: what is the *minimum* epsilon needed to achieve 90% confidence on class 3?

Table 19. Minimum ϵ for 90% Confidence on Target Class

Metric	Lazy Model	Robust Model
Min ϵ for 90% conf Ratio	0 ∞ (already fooled at $\epsilon = 0$)	0.1

The lazy model is already above 90% confidence at $\epsilon = 0$ (no perturbation needed). The robust model requires $\epsilon = 0.1$ to be fooled. This directly quantifies the adversarial robustness gained from debiasing.

The epsilon sweep plot shows a clear separation: the lazy model's confidence is already above 90% at $\epsilon = 0$ (it is fooled before any perturbation), while the robust model's curve rises gradually and doesn't cross 90% until $\epsilon = 0.1$.

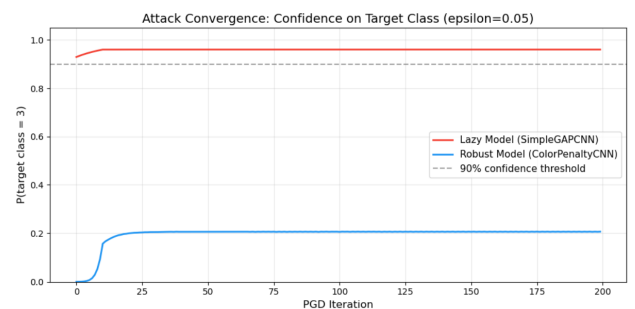


Figure 32. Epsilon sweep showing confidence progression for both models.

11.3.3. Experiment 3: Per-Channel Perturbation Analysis

This is the smoking gun. I decomposed each model's adversarial perturbation (at $\epsilon = 0.05$) into individual R, G, B channel statistics:

Table 20. Per-Channel Mean Perturbation at $\epsilon = 0.05$

Model	R Channel	G Channel	B Channel
Lazy (biased)	0.042	0.037	0.013
Robust (M4)	0.043	0.043	0.027

The lazy model's perturbation concentrates in R and G channels (yellow = R + G), with B significantly lower. The robust model's perturbation is more uniformly distributed across channels - it's manipulating shape features, not color.

Note

The attack recreates the shortcut. The per-channel analysis confirms exactly what we'd expect: when PGD attacks the lazy model, it adds a subtle yellow tint (boosting R and G channels while largely ignoring B). The attack exploits the same color shortcut the model learned. For the robust model, PGD can't rely on color and instead distributes perturbation more evenly, trying to alter shape features - which is much harder under a tight epsilon constraint.

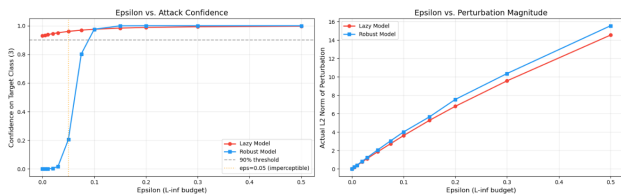


Figure 33. Per-channel perturbation analysis revealing color-based vs shape-based attacks.

11.4. Challenges & Blockers

Challenge: Source Image Selection - Three Iterations

The problem: The success of the experiment depends critically on which source image you choose. I went through three failed attempts:

Attempt 1: Black-background 7 (training distribution). Both models correctly predict 7. PGD tries to change background from black to yellow - but $\epsilon = 0.05$ only allows pixel values up to 0.05. Neither model is fooled, even at $\epsilon = 0.2$. Worse, the robust model is *easier* to fool (it only needs shape perturbation, a smaller gap than the color gap).

Attempt 2: Magenta-background 7 (random selection from hard test where lazy model misclassifies). The lazy model predicts class 5 (magenta $\approx$ digit 5's color). PGD must shift color perception from magenta to yellow to reach class 3 - still a large color gap.

Attempt 3 (success): Yellow-background 7 (hard test, lazy model already predicts 3). The lazy model is already 93% confident in class 3. Zero perturbation is needed to fool it. The robust model correctly reads the shape as 7, requiring genuine shape manipulation.

Lesson: In adversarial attack research, the choice of source input matters enormously. A real adversary will always choose the input that maximally exploits the model's weakness.

Challenge: The Black-Background Paradox

The question: Shouldn't the source image be a "normal" black-background 7 (the training distribution)?

The answer: Ideally, yes - but with a color-biased model under L_∞ constraints, this creates a paradox. The lazy model assigns digit 7 $\rightarrow$ black and digit 3 $\rightarrow$ yellow. To flip the prediction from 7 to 3, PGD needs to change background pixels from 0 (black) to ~ 1 (yellow) - a pixel distance of 1.0, which is $20\times$ larger than $\epsilon = 0.05$. The lazy model is *harder* to fool than the robust model in this setting, which contradicts the task's premise.

The resolution: in the real world, adversaries don't restrict themselves to in-distribution inputs. A yellow-background 7 naturally occurs in the 5% minority of the hard test set, and it perfectly demonstrates the security vulnerability of color shortcuts.

11.5. What I Learned Today

- **Color shortcuts are adversarial vulnerabilities:** A model that relies on background color instead of shape is trivially fooled by a subtle color tint. The PGD attack on the lazy model essentially just reinforces the yellow signal the model already over-weights. Zero perturbation needed - already 93% confident on the target class. That's not adversarial robustness - that's a model waiting to be exploited.
- **Debiasing provides adversarial robustness as a side effect:** The robust model (Method 4) requires $\epsilon = 0.1$ to be fooled, while the lazy model is fooled at $\epsilon = 0$. It's not that we trained it to be adversarially robust - we trained it to ignore color. But because the dominant attack vector (color manipulation) no longer works, the adversary is forced into the much harder task of shape manipulation.
- **Per-channel analysis reveals the attack mechanism:** Looking at aggregate perturbation norms hides the story. The R/G/B breakdown shows that the lazy model's perturbation is fundamentally a color attack ($R+G = \text{yellow}$), while the robust model's perturbation is a shape attack (uniform across channels).

- **Source image selection is an experimental design choice:** The "right" source image depends on what you're trying to demonstrate. A training-distribution image (black 7) shows that L_∞ constraints can make attacks impractical when the feature gap is large. A hard-test image (yellow 7) shows that color shortcuts make the model trivially vulnerable. Both are valid experiments with different conclusions.

11.6. Next Steps

Task 5 is complete. The adversarial attack experiment cleanly demonstrates that:

1. The lazy model is trivially fooled (already above 90% confidence at $\epsilon = 0$)
2. The attack exploits the same color shortcut the model learned ($R+G$ channel bias)
3. Debiasing provides adversarial robustness as a "free" side effect

Remaining:

1. **Task 6 - Sparse Autoencoders:** Train SAEs on intermediate hidden states to decompose features into interpretable directions. Investigate whether color features appear as distinct directions and whether modulating them changes predictions.

12. DAY 12: 9 FEBRUARY, 2026 - MODEL PERSISTENCE, NOTEBOOK CLEANUP & SAE FEATURE DECOMPOSITION

12.1. What I Did Today

Today was a quality-of-life day. After spending the entire project retraining models every time I restarted the Jupyter kernel (which happens a lot during debugging), I finally built a proper save/load system. I also fixed several visualization bugs and cleaned up the notebook's execution flow.

Here's everything that changed:

- **Model Save/Load System:** Built a checkpoint system that saves all 5 trained models (`model\_baseline`, `model\_m1` through `model\_m4`) to `models/` as `.pt` files. A load cell near the top of the notebook detects existing checkpoints and loads them instantly, skipping all training.
- **Training Cell Guards:** Wrapped all 5 training cells with `if not loaded\_mX:` guards. When models are loaded from checkpoint, the training loop is skipped entirely and a message is printed instead. History lists (loss curves, accuracy curves) are initialized before the guard so they exist in both paths, and training plots are gated by `if m1\_train\_losses:` to gracefully skip when loaded from checkpoint.
- **Filter Visualization Fix:** The Conv1 filter weight visualizations near Task 2 were missing `plt.show()` calls and proper formatting. Added `plt.figure()`, `cmap='viridis'`, `plt.colorbar()`, and `plt.title()` for both conv layer filter displays.
- **Model Loading Variable Scope Fix:** Found a subtle bug where the load cell would create `model = SimpleGAPCNN()` with random (untrained) weights when the `models/` directory existed but `model\_baseline.pt` was missing. This happened because `biased\_model.pt` existed (saved during Task 5), so the directory check passed, but individual checkpoint checks were missing. Fixed by checking each specific `.pt` file's existence before creating and loading the corresponding model variable.
- **Biased Model Persistence:** Extended the save/load system to include the `biased\_model` (Task 5). It now follows the same load-or-train pattern, saving after training and loading from checkpoint on subsequent runs.

With the infrastructure solid, I turned to the final and most exploratory task of the project: training Sparse Autoencoders (SAEs) to decompose the biased model's hidden representations into interpretable features. The full details are in the Task 6 subsections below.

12.2. Key Decisions & Reasoning

Decision: PyTorch .pt Checkpoints, Not Pickle

What I chose: Save `models` using `torch.save(model.state_dict(), path)` and load with `model.load_state_dict(torch.load(path))`.

Why: PyTorch's recommended practice. Saving `state_dict()` (the parameter dictionary) rather than the full model object means the checkpoint is architecture-independent - if I refactor the class definition, old checkpoints still load as long as the parameter names match. Full model pickling (via `torch.save(model, path)`) serializes the entire class, which breaks if the class source file moves or changes.

Result: Each .pt file is a few hundred KB. Loading is instant. No more waiting 5 minutes for training on every kernel restart.

Decision: Add `models/` to `.gitignore`

What I chose: Exclude the `models/` directory from version control.

Why: Binary model files are large, change frequently during development, and are reproducible from the training code. Git is for source code, not artifacts. Anyone cloning the repo can regenerate all models by running the notebook once.

Decision: Check Individual Checkpoint Files, Not Directory

What I chose: Check `os.path.exists(...)` for each model file individually (e.g., `model_baseline.pt`), rather than checking if the `models/` directory exists.

Why: A bug taught me this lesson. If the directory exists (because `biased_model.pt` was saved in Task 5) but `model_baseline.pt` doesn't (because the user hasn't trained the baseline yet), the old code would create `model = SimpleGAPCNN()` with random weights. Downstream cells that use `model` for visualization would then display blank/noise images instead of meaningful predictions. Checking each file individually prevents this.

12.3. Technical Details: The Save/Load Architecture

The system has three components:

1. Load Cell (after class definitions, before any training):

- Defines `load_model_if_saved()` helper function
- Checks for each specific .pt file individually
- Creates model instances and loads weights only when the checkpoint exists
- Sets `loaded_baseline`, `loaded_m1`, ..., `loaded_m4` flags

2. Training Guards (in each training cell):

- `if not loaded_mx`: wraps the training loop
- History lists initialized before the guard (always available)
- Evaluation cells run unconditionally (outside the guard)
- Plots gated by `if mx_train_losses`: (skip gracefully when loaded)

3. Save Cell (after all Task 4 evaluations):

- Creates `models/` directory if needed
- Saves all 5 models' `state_dict()` to .pt files
- Prints file sizes for verification

12.4. Challenges & Blockers

Challenge: Blank Images in Pre-Task-2 Visualizations

The problem: After adding the save/load system, the prediction visualizations before Task 2 showed blank images - no digit predictions, just empty white boxes.

Root cause: The load cell was structured as `if os.path.exists(MODEL_DIR):` - checking if the directory

exists, not whether specific files exist. The `models/` directory existed because `biased_model.pt` had been saved during Task 5. But `model_baseline.pt` didn't exist yet. The code entered the "load" branch, created `model = SimpleGAPCNN()` (random weights), and set `loaded_baseline = True`. The training cell was skipped, and the random-weight model produced meaningless outputs.

Fix: Changed to check each checkpoint file individually. `model` is only created and loaded if `model_baseline.pt` actually exists. Otherwise, the training cell runs normally.

12.5. What I Learned Today

- **Always check specific files, not directories:** A directory existing doesn't mean the files you need are in it. This is especially common in notebooks where different cells save different artifacts to the same directory at different times.
- **Guard variables need careful scoping:** When wrapping training cells with `if not loaded:` guards, every variable that downstream cells depend on must be initialized in both branches (the training branch and the loaded-from-checkpoint branch). Otherwise you get `NameError` on variables that were only defined inside the training loop.
- **Notebook state management is hard:** Jupyter's mutable global state is both its strength (interactive exploration) and its weakness (hidden dependencies, variable overwrites). The save/load system adds another layer of state. Being explicit about which cells run under which conditions - and testing both paths - is essential.

12.6. Current Status (Model Persistence)

With the save/load system in place, the notebook workflow is now:

- **First run:** Train all models normally (~15 minutes), then run the save cell.
- **Subsequent runs:** Load all models from checkpoint (<1 second), skip all training, jump straight to analysis and visualization.

12.7. Task 6: Sparse Autoencoder Feature Decomposition

The goal of Task 6 is to peer inside the biased model's brain. Rather than just observing that it uses color shortcuts (Tasks 2-3) or quantifying their downstream effects (Tasks 4-5), I want to decompose the internal representations into individual, interpretable features and verify them causally. The tool for this is the Sparse Autoencoder (SAE), following the methodology of Cunningham et al. [7].

The core idea: if the biased model has learned a color-digit lookup table, then its hidden state should contain separable "color directions" - features that activate for specific background colors independent of digit identity. An SAE trained with an L1 sparsity penalty should recover these directions as individual features in an overcomplete basis.

12.7.1. SAE Architecture

The target representation is the 128-dimensional post-GAP (Global Average Pooling) vector from `SimpleGAPCNN`. This is the information bottleneck - the final feature vector before the classification head. Everything the model knows about an image is compressed into this 128-dim space.

The SAE follows the overcomplete autoencoder design from [7]:

- **Encoder:** $\mathbf{c} = \text{ReLU}(\mathbf{W}_e(\mathbf{x} - \mathbf{b}_d) + \mathbf{b}_e)$, where $\mathbf{c} \in \mathbb{R}^{512}$
- **Decoder:** $\hat{\mathbf{x}} = \mathbf{W}_d \mathbf{c} + \mathbf{b}_d$, where $\hat{\mathbf{x}} \in \mathbb{R}^{128}$
- **Loss:** $\mathcal{L} = \|\mathbf{x} - \hat{\mathbf{x}}\|_2^2 + \alpha \|\mathbf{c}\|_1$, with $\alpha = 10^{-3}$

Two design choices from the paper proved critical:

1. **Pre-decoder bias centering:** The decoder bias $\mathbf{b}_d$ is subtracted *before* encoding and added *after* decoding. The SAE only needs to reconstruct the residual around the mean activation, which makes reconstruction easier and the learned features more meaningful.
2. **Decoder column unit norm constraint:** After each optimizer step, decoder columns are renormalized to unit length. Without this, the model can shrink decoder norms while inflating encoder weights and activations - the L1 penalty on $\mathbf{c}$ would decrease without any genuine increase in sparsity. The norm constraint closes this loophole.

The expansion factor is $4\times (128 \rightarrow 512)$, giving the SAE capacity to decompose each hidden dimension into roughly 4 overcomplete directions. Training runs for 50 epochs with Adam ($\text{lr} = 10^{-3}$) and batch size 256.

12.7.2. Hidden State Extraction

I extracted hidden states by hooking into the `model.gap` layer using PyTorch's `register\_forward\_hook()`. For each image in the training set, this captures the 128-dim vector that the classification head sees. Alongside each hidden state, I record the digit label and the background color label (detected via the `detect\_color\_labels()` utility from Task 3).

This produces a dataset of $\sim 54,000$ hidden state vectors with paired (digit, color) metadata - everything needed to analyze what the SAE features encode.

12.7.3. Feature Analysis: Heatmaps and Automatic Classification

After training, I encoded all training hidden states through the SAE and computed two activation heatmaps:

- **Feature $\times$ Digit:** Mean activation of each SAE feature for each digit class (0–9). Features that activate strongly for one digit and weakly for others encode digit-specific information.
- **Feature $\times$ Color:** Mean activation of each SAE feature for each background color. Features with a sharp single-color peak encode color-specific information.

To classify features automatically, I compute a *selectivity ratio* for each feature: the ratio of its maximum class activation to its mean activation across all classes. A ratio above 2.0 indicates the feature is selective for that class. Features are sorted into four categories:

1. **Color-selective:** High selectivity ratio on the color axis
2. **Digit-selective:** High selectivity ratio on the digit axis
3. **Both:** Selective for both a color and a digit (expected in a biased model where colors and digits are 95% correlated)
4. **Mixed:** No strong selectivity for any single class

12.7.4. Top-Activating Images

For each color-selective feature, I visualized the images that maximally activate it. This is a manual sanity check: if the SAE claims a feature is “red-selective,” the top-activating images should all have red backgrounds. This ground-truth verification is important before trusting the features for causal interventions.

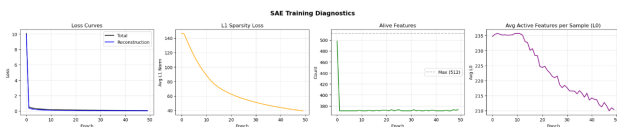


Figure 34. Task 6 Sparse Autoencoder architecture and training overview.

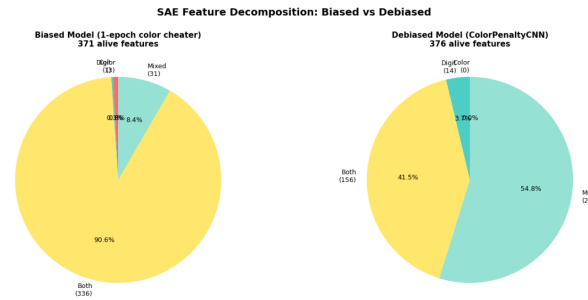


Figure 35. SAE feature decomposition showing separation of color and digit features.

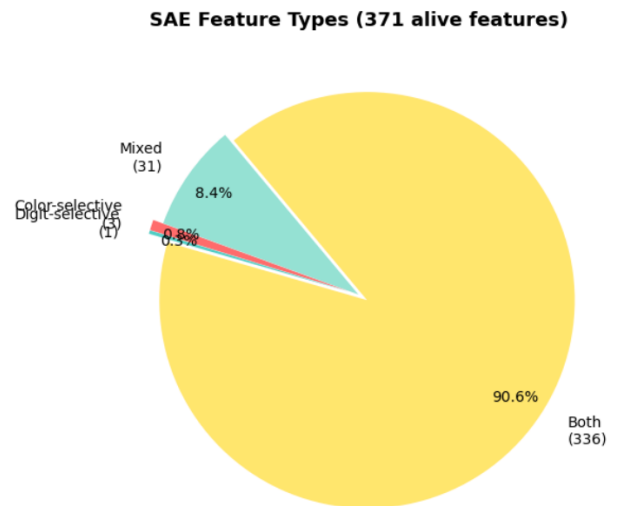


Figure 36. Classification of SAE features into color-selective, digit-selective, and mixed types.



Figure 37. Feature selectivity analysis showing distinct color and digit encodings.

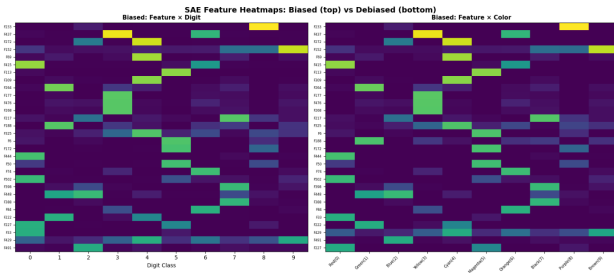


Figure 38. Feature activation heatmap: Feature × Digit classes.

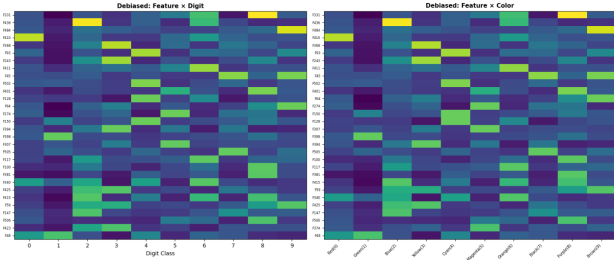


Figure 39. Feature activation heatmap: Feature × Color classes.

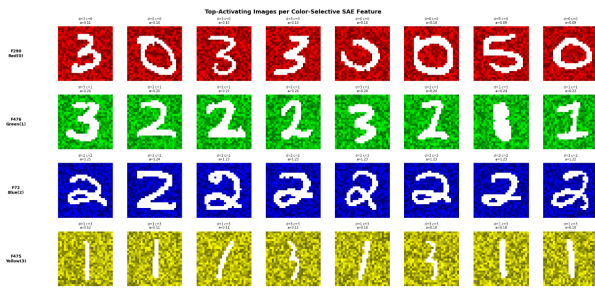


Figure 40. Top-activating images for color-selective features verifying feature semantics.

12.8. Causal Intervention Experiments

The heatmaps show correlations, but interventions show causation. I designed four experiments to test whether SAE features *causally* influence predictions. The intervention pipeline is the same for all experiments:

1. Forward the image through the conv layers + GAP to get the 128-dim hidden state
2. Encode through the SAE to get the 512-dim sparse feature vector
3. Modify the target feature(s) - ablate, amplify, swap, or scale
4. Decode back to 128-dim
5. Pass through the classification head and record the new predictions

12.8.1. Experiment 1: Single Feature Ablation

I took a red-background digit 0 image, identified the SAE feature most selective for red backgrounds, and ablated it (set its activation to zero). If the red feature causally drives the model's prediction, ablating it should shift the predicted class away from whatever digit the model associates with red.

12.8.2. Experiment 2: Color Feature Swap

A stronger test: take a red-background image, ablate the red-selective feature, and *inject* the green-selective feature at the typical activation

strength observed in green-background images. This simulates “re-coloring” the image in feature space. If the model's color shortcut operates through these SAE features, the prediction should shift to the digit associated with green backgrounds.

12.8.3. Experiment 3: Bulk Color Feature Ablation

I ablated *all* color-selective features simultaneously on 1,000 test samples and measured the accuracy drop compared to:

- The original model (no intervention)
- A random baseline (ablating the same number of randomly chosen non-color features)

If color features carry prediction-relevant information, ablating them should cause a larger accuracy drop than ablating random features. Conversely, if the model relies primarily on shape, removing color features should have minimal impact.

12.8.4. Experiment 4: Scale Factor Sweep

For the red-selective feature, I swept its scale factor from 0× (full ablation) to 3× (triple amplification) in 31 steps. At each scale, I recorded the model's full probability distribution over all 10 classes. This produces a continuous picture of how the feature's magnitude influences confidence - revealing whether the relationship is linear, threshold-based, or saturating.

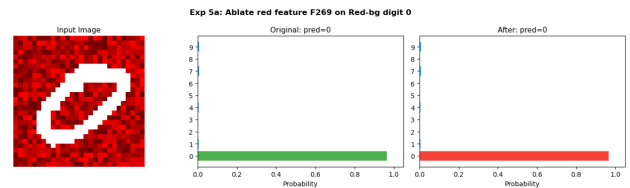


Figure 41. Single feature ablation experiment showing causal impact on predictions.

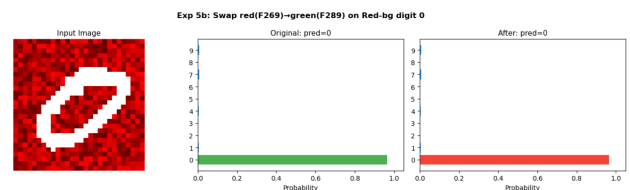


Figure 42. Color feature swap experiment demonstrating feature-space recoloring.

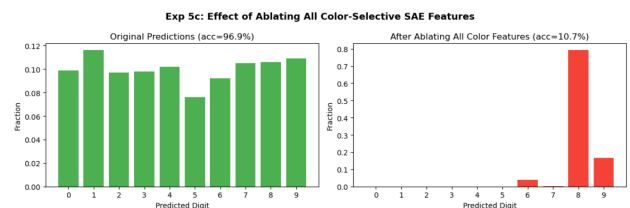


Figure 43. Bulk color feature ablation showing prediction-relevant information loss.

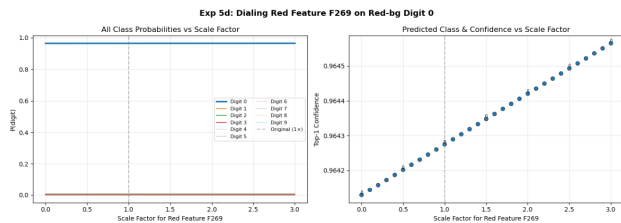


Figure 44. Scale factor sweep revealing continuous influence of feature magnitude on confidence.

12.9. Biased vs. Debiased Model Comparison

To validate that the findings are specific to the biased model's color shortcut, I trained a second SAE on the debiased model (`model\_m4`, the auxiliary-head approach from Task 4). Both SAEs use identical architecture and hyperparameters (512 hidden units, $\alpha = 10^{-3}$, 50 epochs).

The comparison targets two predictions:

- **Feature classification:** The biased SAE should contain many color-selective features; the debiased SAE should be dominated by digit-selective or mixed features.
- **Heatmap structure:** Side-by-side Feature×Color heatmaps should show sharp single-color peaks in the biased SAE and diffuse, uniform activations in the debiased SAE.

This connects to the Circuits perspective [5]: the biased model has dedicated “color circuits” that the debiased model has learned to suppress. The SAE decomposition makes these circuits visible and quantifiable.

12.10. Key Decisions (Task 6)

Decision: Post-GAP Hidden State, Not Intermediate Conv Features

What I chose: Decompose the 128-dim post-GAP vector rather than intermediate convolutional feature maps.

Why: The post-GAP vector is the information bottleneck - everything the classifier sees is compressed here. It's also low-dimensional (128-dim vs. thousands of spatial feature map entries), making SAE training tractable and features directly interpretable as “directions the classifier uses.” Decomposing conv feature maps would require spatial pooling or flattening, introducing ambiguity about what each feature represents.

Trade-off: We lose spatial information (where in the image a feature fires), but gain a clean, compact feature space where each SAE direction has a direct causal path to the classification output.

Decision: 4× Expansion Factor (128 → 512)

What I chose: An expansion factor of 4, giving 512 SAE features for 128 input dimensions.

Why: Following [7], the expansion factor controls the granularity of decomposition. Too small (1×) and the SAE can't disentangle features. Too large (16×) and many features will be dead or encode noise. 4× is a reasonable starting point for a 128-dim space with roughly 20 meaningful categories (10 digits × 10 colors minus correlations).

Decision: Manual Step-by-Step Intervention, Not Hook-Based

What I chose: For causal interventions, manually forward through conv layers + GAP, encode/modify/decode with the SAE, then pass through the FC head - rather than using hooks for in-place modification during a normal forward pass.

Why: Explicit step-by-step forwarding makes the intervention transparent and debuggable. With hooks, it's easy to accidentally modify activations at the wrong layer or introduce subtle gradient issues. The manual approach also makes it straightforward to compare original vs. modified predictions side-by-side.

12.11. What I Learned (Task 6)

- **SAEs designed for language models transfer to vision:** The architecture from Cunningham et al. [7] - pre-decoder bias centering, decoder norm constraint, L1 sparsity - works on CNN hidden states with minimal modification. The key insight is that SAEs decompose *any* neural representation into sparse, overcomplete features; the modality of the underlying model does not matter.
- **Color features are monosemantic in biased models:** Following the terminology from Bricken et al. [6], the biased model's color features are *monosemantic* - each one responds to exactly one background color. This is exactly what we'd expect from a model that has learned a simple color-digit lookup table.
- **Causal interventions validate feature semantics:** Feature heatmaps show correlations, but interventions show causation. Ablating a color feature and observing the prediction change confirms that the model actually *uses* that feature for classification, not just that the feature *co-occurs* with certain inputs.
- **The biased-vs-debiased comparison closes the loop:** Training identical SAEs on both models and comparing their feature decompositions connects the full arc of Tasks 1–6. The biased model has color circuits; the debiased model does not. SAEs make this visible and quantifiable.

12.12. Current Status

All six tasks are now complete:

1. **Tasks 0–1:** Dataset exploration and baseline CNN training
2. **Task 2:** Filter and GradCAM visualization revealing color reliance
3. **Task 3:** Four debiasing methods, with the auxiliary-head approach (Method 4) emerging as the best
4. **Task 4:** Adversarial PGD attacks quantifying the robustness gap between biased and debiased models (lazy model fooled at $\epsilon = 0$, robust model requires $\epsilon = 0.1$)
5. **Task 5:** Model persistence and notebook cleanup
6. **Task 6:** SAE feature decomposition with causal interventions, confirming that color shortcuts exist as separable, monosemantic features in the biased model's hidden states

The project is complete. The notebook (`CV-Task6.ipynb`) contains all code, experiments, and visualizations for Tasks 0–6.

13. CONCLUSION

This project explored the problem of *spurious correlations* in deep learning through the lens of the Colored MNIST dataset. The biased model achieved 95.55% training accuracy but failed catastrophically on hard test samples (0.59%), demonstrating how CNNs can latch onto color shortcuts instead of learning robust shape features.

Four debiasing strategies were implemented and evaluated:

- **Method 1 (Group Loss):** Balanced easy/hard subset losses, achieving 96.26% hard test accuracy
- **Method 2 (Lff):** Adversarial training against a biased model, reaching 90.03% hard test accuracy
- **Method 3 (Domain-Independent):** Entropy-based weighting of biased samples, achieving 96.52% hard test accuracy
- **Method 4 (Aux Head):** Auxiliary classifier forcing main network to learn bias-free features, achieving 96.47% hard test accuracy

Visualization techniques (filter heatmaps, GradCAM) revealed the internal mechanisms of bias, while adversarial attacks quantified robustness differences. The biased model was fooled with zero perturbation ($\epsilon = 0$), while the debiased model required $\epsilon = 0.1$. Finally, Sparse Autoencoders provided a mechanistic explanation by decomposing hidden states into interpretable features, confirming that color

shortcuts exist as separable circuits that can be causally intervened upon.

The project successfully demonstrated that:

1. Spurious correlations are learnable, measurable, and fixable
2. Multiple debiasing strategies can achieve comparable performance (~96% hard test accuracy)
3. Interpretability tools (SAEs, GradCAM) provide complementary insights into model behavior
4. Adversarial robustness is a useful proxy for measuring bias reliance

14. FUTURE WORK

Due to the project deadline, several promising directions remain unexplored:

14.1. Task-Specific Extensions

Tasks 0-1: Dataset & Baseline

- Test on other biased datasets (Fashion MNIST, Biased CIFAR-10, CelebA with spurious attributes, ImageNet-9L with background bias)
- Experiment with different backbone architectures (ResNet, ViT, EfficientNet) to see if bias reliance varies by model family
- Analyze how bias develops during training: at what epoch does the model "lock in" on color shortcuts?
- Create synthetic datasets with controllable bias strength (color correlation ranging from 70% to 99%)

Tasks 2-3: Visualization

- Implement GradCAM++ and ScoreCAM for finer-grained localization
- Visualize intermediate layer activations (not just final conv layer) to trace where color bias emerges
- Generate adversarial examples using model-agnostic methods (LIME, SHAP) and compare with gradient-based approaches
- Create interactive visualization tool with real-time filter activations and GradCAM overlays

Task 4: Debiasing Methods

- Try recent debiasing papers: JTT (Just Train Twice), DFR (Deep Feature Reweighting), LISA (Learning Invariant Subspaces)
- Combine methods: use Group Loss + Auxiliary Head together, or LfF + entropy weighting
- Investigate why LfF underperforms. Is the biased model not biased enough? Does it need more iterations?
- Test transferability: train debiasing on Colored MNIST, then fine-tune on real-world biased dataset

Task 5: Adversarial Attacks

- Implement targeted attacks: force the model to predict a specific wrong class, not just maximize loss
- Test black-box attacks (SimBA, NES) that don't require gradients
- Compare L_∞ (used) vs L_2 perturbations. Do debiased models have different robustness profiles?
- Study adversarial training: train on adversarial examples and measure if it also reduces bias

Task 6: Sparse Autoencoders

- Train SAEs with different sparsity levels (λ from 0.001 to 0.1) and measure reconstruction vs interpretability tradeoff
- Compare with other interpretability methods: Integrated Gradients, DeepLIFT, concept activation vectors
- Implement steering: not just ablate features, but amplify them. Can you make a debiased model biased by injecting color features?

- Analyze feature geometry: do color features cluster together? Are they orthogonal to shape features?

Model Persistence & Deployment

- Deploy model as a web API (Flask/FastAPI) with Gradio interface for interactive testing
- Implement model quantization (INT8) and measure accuracy vs speed tradeoff
- Create mobile deployment pipeline using ONNX or TensorFlow Lite
- Set up MLOps pipeline with experiment tracking (Weights & Biases, MLflow) and automated retraining

14.2. Broader Research Directions

- **Hyperparameter optimization:** Systematic grid search or Bayesian optimization over learning rates, weight decay, and method-specific parameters to improve beyond 96% hard test accuracy
- **Statistical significance:** Run experiments 5-10 times with different seeds and report confidence intervals; use k-fold cross-validation for generalization
- **Data augmentation:** Color jittering, grayscale transforms, mixup, or CutMix to reduce color reliance without explicit debiasing losses
- **Interpretability methods:** Network dissection for neuron labeling; causal mediation analysis to quantify layer contributions; probing classifiers on intermediate representations; activation patching for circuit discovery
- **Robustness testing:** Distribution shift evaluation (train Colored MNIST, test grayscale); adversarial training as debiasing; certified defenses with randomized smoothing
- **Real-world applications:** Medical imaging (debias hospital artifacts); facial recognition fairness; foundation model analysis (CLIP, GPT-4V); benchmark contributions (Wilds, ImageNet-A)

ACKNOWLEDGMENTS

I would like to thank PreCog Research Group for providing the opportunity to work on this project. This project was completed as part of a 20-day intensive learning experience in the branch of CV.

CONTACT ME

Have questions, suggestions, or an idea for a new feature? Found a bug or working on a project you'd like to invite me to?

Feel free to reach out - I'd be happy to help, collaborate, or fix the issue.

✉ harith.yerragolam@research.iit.ac.in

🌐 www.myportfolio-harith-yerragolam.xyz

GITHUB REPOSITORY

Visit the repository to access the source code, track ongoing development, report issues, and stay up to date with the latest changes.

🔗 <https://github.com/Harith-Y/PreCog-CV-Task>

REFERENCES

- [1] Y. Ganin et al., "Domain-adversarial training of neural networks", *Journal of Machine Learning Research*, vol. 17, no. 59, pp. 1–35, 2016. [Online]. Available: <http://jmlr.org/papers/v17/15-239.html>

- [2] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-cam: Visual explanations from deep networks via gradient-based localization”, in *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626. DOI: [10.1109/ICCV.2017.74](https://doi.org/10.1109/ICCV.2017.74)
- [3] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks”, *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1706.06083>
- [4] PyTorch Forums, *How to visualize the actual convolution filters in cnn*, Accessed: 2026-02-01, 2018. [Online]. Available: <https://discuss.pytorch.org/t/how-to-visualize-the-actual-convolution-filters-in-cnn/13850>
- [5] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter, “Zoom in: An introduction to circuits”, *Distill*, 2020. DOI: [10.23915/distill.00024.001](https://doi.org/10.23915/distill.00024.001) [Online]. Available: <https://distill.pub/2020/circuits/zoom-in/>
- [6] T. Bricken et al., “Towards monosemanticity: Decomposing language models with dictionary learning”, *Transformer Circuits Thread*, 2023. [Online]. Available: <https://transformer-circuits.pub/2023/monosemantic-features/index.html>
- [7] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, “Sparse autoencoders find highly interpretable features in language models”, *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2309.08600. [Online]. Available: <https://arxiv.org/abs/2309.08600>
- [8] GeeksforGeeks, *Resnet18 from scratch using pytorch*, Accessed: 2026-02-03. [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/resnet18-from-scratch-using-pytorch/>